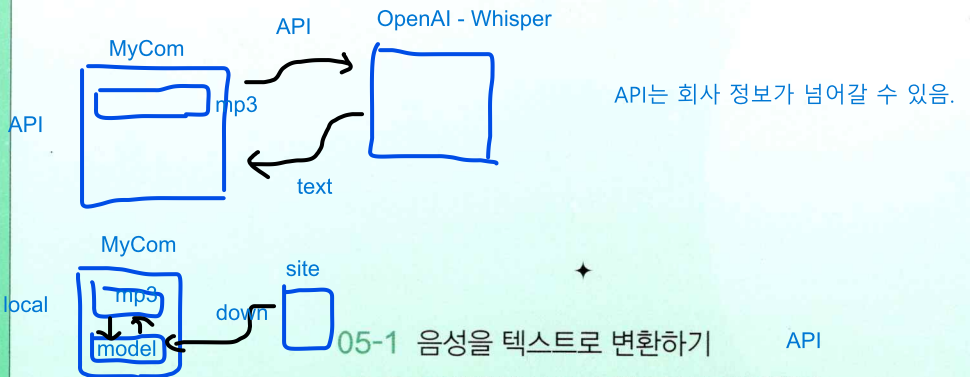


회의록을 정리하는 AI 서기

학교나 직장에서는 회의를 마친 후 회의록을 작성해 공유하는 경우가 많습니다. 회의하는 동안 누군가 내용을 기록하고 요약해 전달해 준다면 좋겠죠. 이번 장에서는 회의할 때 녹음한 파일을 회의록으로 정리해 주는 AI 서기 프로그램을 만들어 보겠습니다.

자동



05-1 음성을 텍스트로 변환하기 API

05-2 로컬에서 음성을 텍스트로 변환하기 on-Premise

05-3 문장과 화자 구분하기

05-4 회의록을 정리하는 AI 서기 완성하기

05-1 음성을 텍스트로 변환하기

바로 LLM에 넣을 수 없음.

cf> TTS : Text To Speech

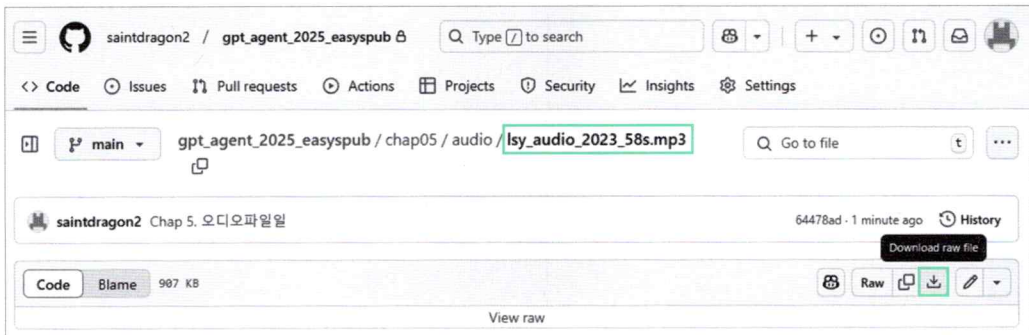
GPT는 텍스트를 이해하고 처리하는 언어 모델이므로 녹음된 음성 파일을 텍스트로 변환해서 제공해야만 처리할 수 있습니다. 음성을 텍스트로 변환하는 기술을 **STT^{Speech-To-Text}**라고 합니다. 오픈AI는 **위스퍼^{Whisper}** API를 통해 이 기능을 제공합니다. 오픈AI의 공식 문서에 따르면 위스퍼 API의 요금은 분당 6센트(2025년 4월 기준)이며 한국 원화로 계산하면 1분에 10원도 안 되는 비용이므로 충분히 사용할 가치가 있습니다.

Do it! 실습 위스퍼 API 활용하기

■ 결과 파일: chap05/sec01/openai_api_whisper.ipynb

텍스트로 변환할 음성이 담긴 MP3 파일을 준비합니다. 여러분이 원하는 **MP3 파일을 사용**하면 됩니다. 예제와 같은 파일로 작업하고 싶다면 다음 링크에서 파일을 내려받으세요. 이 파일은 제가 과거에 만든 동영상 강좌에서 앞부분 58초를 추출한 것입니다.

✦ MP3 파일 내려받기: https://github.com/saintdragon2/gpt_agent_2025_easyspub/blob/main/chap05/audio/lisy_audio_2023_58s.mp3

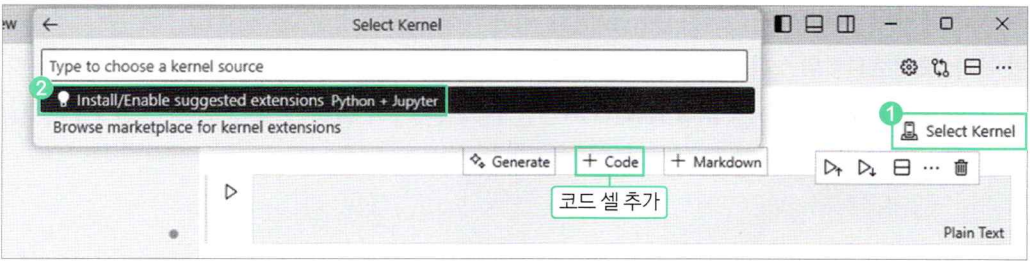


주피터 노트북 사용하기

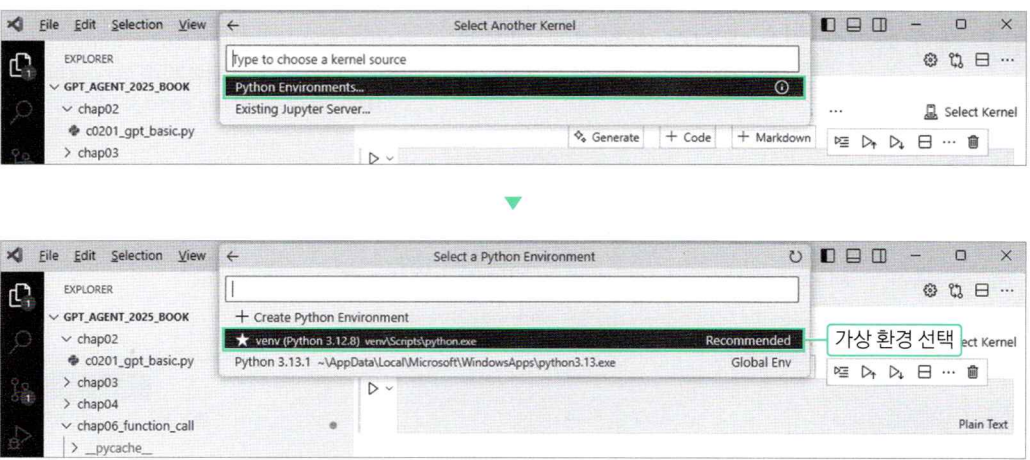
단계별로 기능을 익히기 위해 주피터 노트북 파일을 사용하겠습니다. **VS Code**를 사용하고 있다면 확장자가 **.ipynb**인 파일을 생성하기만 하면 됩니다.

ipykrenel 설치 필요

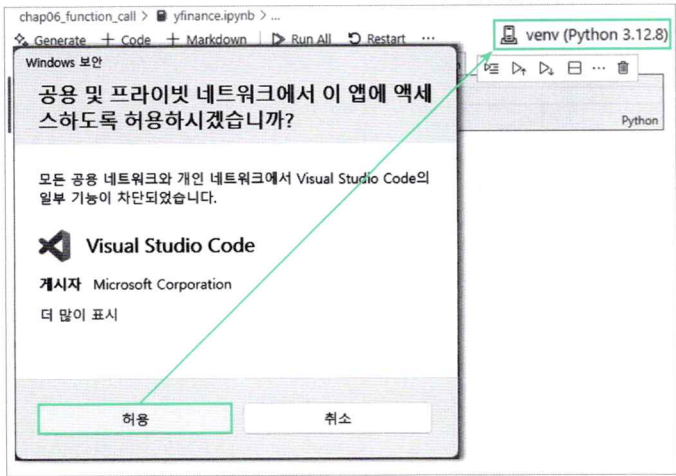
1. chap05 폴더를 만들고 그 안에 openai_api_whisper.ipynb 파일을 만듭니다. 이 파일을 열면 코드 셀(Code Cell)을 추가할 수 있는 [+Code(코드)] 버튼이 있고, 오른쪽 상단에는 [Select Kernel(커널 선택)] 버튼이 있습니다. 만약 이 프로젝트 폴더에서 ipynb 파일을 처음 사용한다면 [Select Kernel] 버튼을 클릭하고 표시되는 화면에서 파이썬과 주피터를 사용할 수 있는 익스텐션을 설치합니다.



2. 익스텐션을 설치하면 파이썬 환경을 선택하는 메뉴가 나옵니다. 여기서 [Python Environments...]를 클릭하고 현재 프로젝트 폴더의 가상 환경인 venv를 선택합니다.



3. 주피터 노트북을 위한 커널을 처음 선택한다면 다음처럼 보안 경고가 나올 수 있습니다. [허용]을 클릭하면 다음처럼 주피터 노트북에서 사용할 가상 환경 venv가 자동으로 선택됩니다.



이제 파이썬 코드를 입력하고 실행해 보시다.

4. [+Code] 버튼을 클릭하면 파이썬 코드를 입력할 수 있는 셀이 나옵니다. 이 셀에 테스트를 위한 파이썬 코드를 입력합니다. 여기에서는 다음과 같이 오픈AI의 API 클라이언트를 선언하는 코드를 사용했습니다.

◆ 주피터 노트북 결과 파일을 확인할 때는 코드 박스의 셀 번호를 참고하세요.

테스트 코드 입력하기

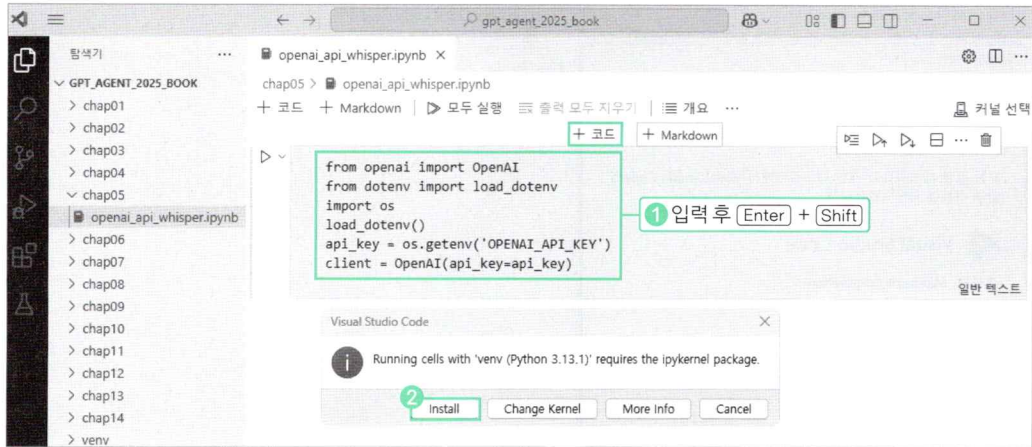
openai_api_whisper.ipynb
(1)

```

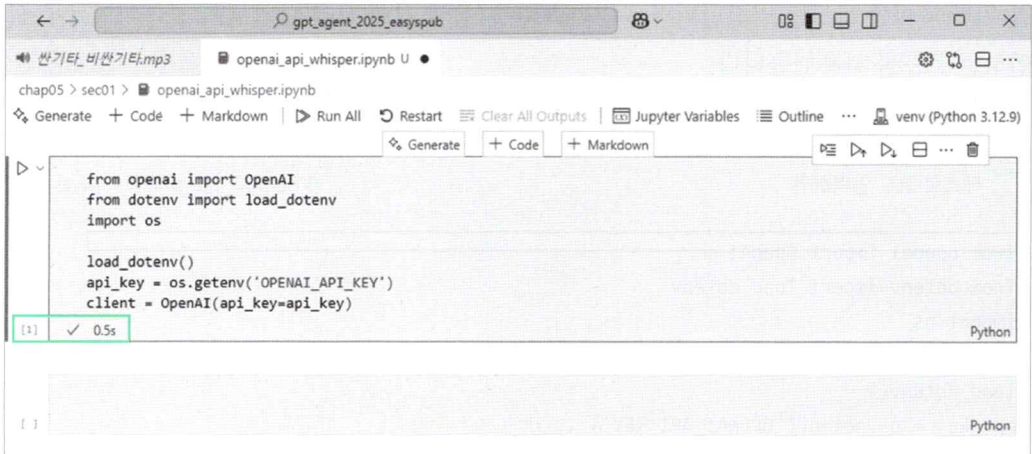
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
api_key = os.getenv('OPENAI_API_KEY')
client = OpenAI(api_key=api_key)
    
```

5. 그리고 **(Shift) + (Enter)**를 눌러 실행하면 ipykernel 패키지가 필요하다는 메시지가 나옵니다. [Install] 버튼을 클릭해 설치합니다.



설치를 완료한 후 실행해 보면 다음처럼 파이썬 코드가 잘 실행됩니다. 셀의 왼쪽 아래에 넘버링이 표시되고 체크 ☒ 표시가 생기면 셀 실행이 성공했다는 의미입니다. 이제 이 셀에 파이썬 코드를 입력하며 진행할 수 있습니다.



MP3 파일을 텍스트로 변환하기

다음은 오픈AI에서 제공하는 위스퍼 API 문서를 바탕으로 코드를 작성해보겠습니다.

1. 우선 오픈AI API를 사용하기 위해 **client**를 선언합니다. 앞에서 주피터 노트북을 설정할 때 사용한 코드와 같습니다.

```
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
api_key = os.getenv('OPENAI_API_KEY')
client = OpenAI(api_key=api_key)
```

2. 오픈AI의 STT 관련 문서를 참고하여 다음처럼 작성합니다. `with open`을 사용하여 파일을 `audio_file`로 엽니다. 그 내용을 `client.audio.transcriptions.create` 함수로 오픈AI 서버에 전송하면 음성을 텍스트로 변환하여 결과를 반환합니다.

◆ 오픈AI의 STT 관련 문서: <https://platform.openai.com/docs/guides/speech-to-text>

```
audio_file_path = './audio/lsy_audio_2023_58s.mp3' # MP3 파일 경로 입력

with open(audio_file_path, 'rb') as audio_file:
    transcription = client.audio.transcriptions.create(
        model="whisper-1",
        file=audio_file
    )

transcription
```

이 셀을 실행하면 다음과 같이 음성 파일이 텍스트로 출력됩니다. ‘챗GPT’를 ‘채GPT’로 오인해서 아쉽지만 이 정도면 전반적으로 쓸 만합니다.

Transcription(text='안녕하세요. 이 강의는 GPT API로 챗봇 만들기라는 내용을 다루는 강의입니다. GPT API에 대해서 생소하신 분들도 있을 텐데 우리가 잘 알고 있는 채GPT, 채GPT 기능을 이용해서 우리가 원하는 프로그램을 어떻게 만드는데 대해서 이야기할 거예요. 그래서 뭐 이런 강의들이 사실 많이 있습니다. 그래서 여러 가지들이 있는데 좀 이 강의의 특징이라고 한다면 GPT로 명확한 미션을 달성하는 챗봇 프로그램을 만드는 게 사실 쉽지는 않은데 이걸 어떻게 해서 구현을 하는지 그리고 그게 왜 필요한지에 대해서 좀 이야기를 할 거고요. 그 예제로 예제는 여러 가지가 될 수 있는데 여기서 예제로 하는 것은 음악 플레이리스트 동영상상을 자동으로 대화를 통해서 생성하는 프로그램 만드는 것을 다루려고 합니다. 그래서 프로그램이 실행되는 모습을 한번 보여드릴게요. 우리가 만들 프로그램은 이런 식으로 이제 나타나게 되고')

★ 한 걸음 더!

위스퍼 API로 한국어 음성 파일을 영어로 바로 번역하기

위스퍼 API는 번역 기능도 제공합니다. 새로운 셀에 앞에서 만든 코드 내용을 복사한 뒤 `client.audio.transcriptions`라고 되어 있던 부분을 `client.audio.translations`로 수정하면 됩니다.



MP3의 한국어 음성을 영어로 번역하기

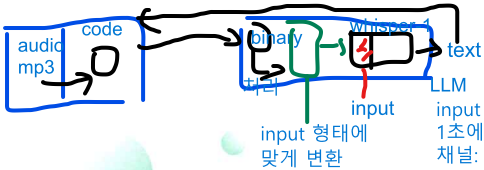
openai_api_whisper.ipynb (3)

```
with open(audio_file_path, 'rb') as audio_file:
    transcription = client.audio.translations.create(
        model="whisper-1",
        file=audio_file,
    )

transcription
```

코드를 실행해 보면 그럴싸하게 영어로 잘 번역했습니다! 물론 한국어로 받아쓴 내용을 다시 GPT-4o나 GPT-4o mini와 같은 언어 모델을 사용해 번역할 수도 있습니다. 하지만 위스퍼 API를 이용하면 오픈AI API를 이중으로 사용하지 않아 토큰 사용량과 시간을 절약할 수 있습니다.

```
Translation(text="Hello, this is a lecture on how to make a chatbot with GPT API.
Some of you may be unfamiliar with GPT API. We're going to talk about how to make the
program we want using the chat GPT function that we know well. So there are a lot of
lectures like this. There are many things, but if I were to say the characteristics
of this lecture, it's not easy to make a chatbot program that achieves a clear mis-
sion with GPT. I'm going to talk about how to implement this and why it's necessary.
As an example, there can be many examples. The example here is to create a program
that automatically creates a music playlist video through conversation. So let me
show you how the program runs. The program we're going to make is going to look like
this.")
```



Local 변환 => ffmpeg -> 세팅
Model => 허깅페이스 -> whisper model

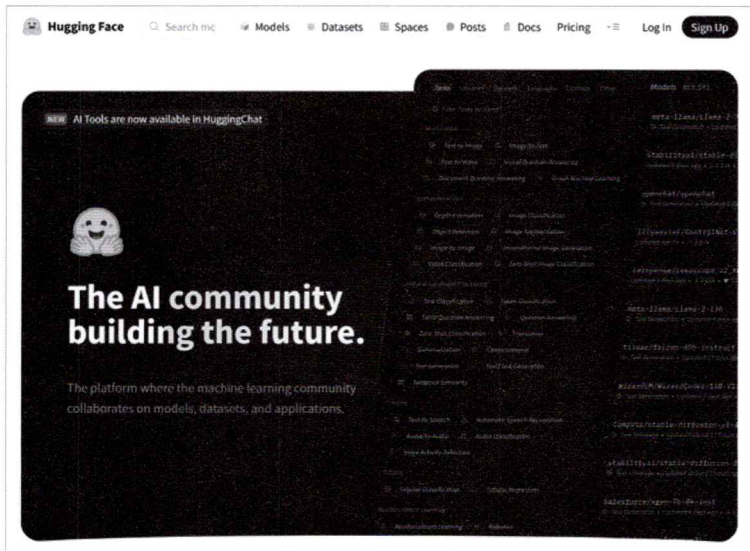
다운로드, 권한..

05-2 로컬에서 음성을 텍스트로 변환하기

오픈AI의 위스퍼 API를 활용하여 음성 파일을 텍스트로 변환하면 음성 파일의 내용이 오픈AI 서버로 전송되어 처리된 후 반환되므로 컴퓨터의 성능에 관계없이 간단한 코드 몇 줄로 기능을 구현할 수 있습니다. 그러나 회의 내용이 회사 밖으로 유출되면 안되거나 음성 파일의 길이가 길어 처리 비용이 우려된다면 로컬 환경에서 언어 모델을 내려받아 처리할 수 있습니다.

허깅페이스 모델 다운받기 위해 들어감.

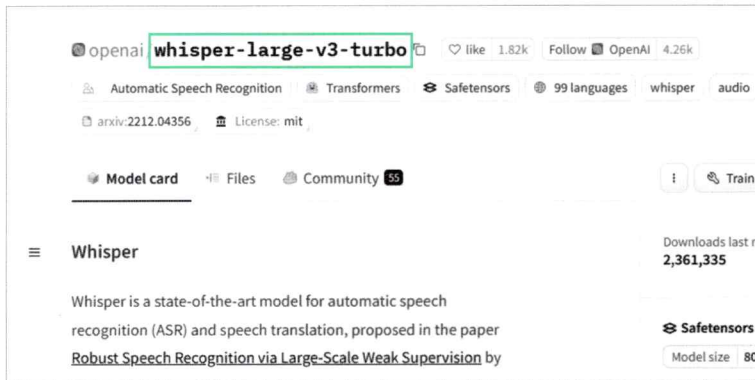
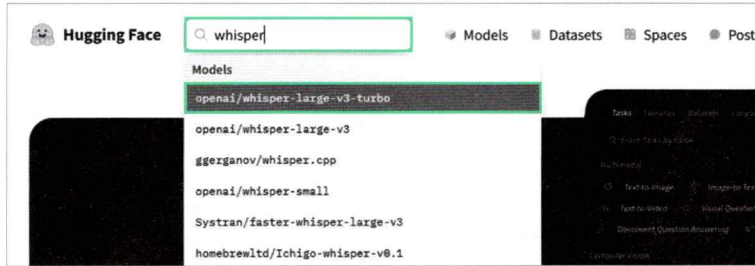
허깅페이스 Hugging Face는 인공지능 모델을 개발하는 회사로, 허깅페이스 플랫폼 서비스로 더욱 잘 알려졌습니다. 허깅페이스 플랫폼에서는 전 세계의 수많은 개발자와 연구자가 만든 인공지능 모델, 데이터셋, 문서를 서비스합니다.



허깅페이스 웹 사이트

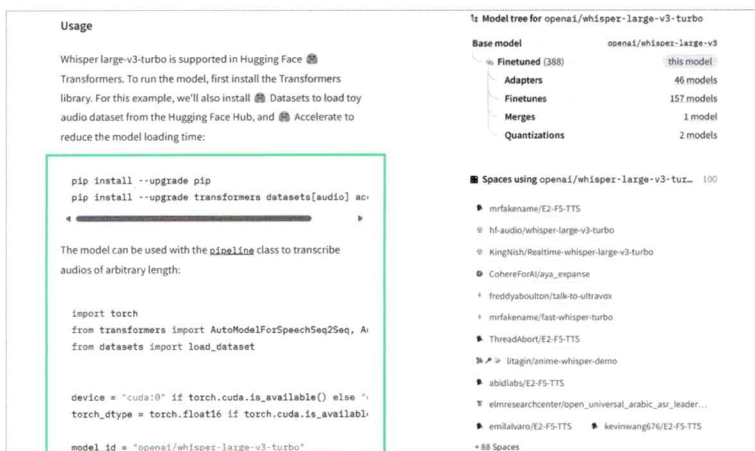
앞서 실습에서 활용한 오픈AI에서 API로 제공하는 위스퍼 모델 역시 허깅페이스에 공개되어 있습니다. 허깅페이스 웹 사이트(<https://huggingface.co>)에 접속해 왼쪽 위 검색 창에서 'whisper'를 검색해 보세요. 여기에서는 whisper-large-v3-turbo 모델을 선택했습니다.

학습이 끝난 모델



허깅페이스에 공개된 whisper-large-v3-turbo 모델

페이지를 아래로 내리면 위스퍼 모델의 사용법이 있습니다. 이어지는 실습에서 단계별로 따라가 봅시다.



Do it! 실습 위스퍼 모델을 내려받아 로컬에서 사용하기

■ 결과 파일: sec02/huggingface_whisper.ipynb

1. 단계별로 따라 하기 위해 주피터 노트북 파일을 만듭니다. 파일명은 huggingface_whisper.ipynb로 정했습니다. 허깅페이스의 문서를 보면 먼저 `pip install`로 필요한 패키지를 설치하도록 되어 있습니다. 첫 번째 셀을 만들고 패키지를 설치합니다.

필요한 패키지 설치하기 (1)

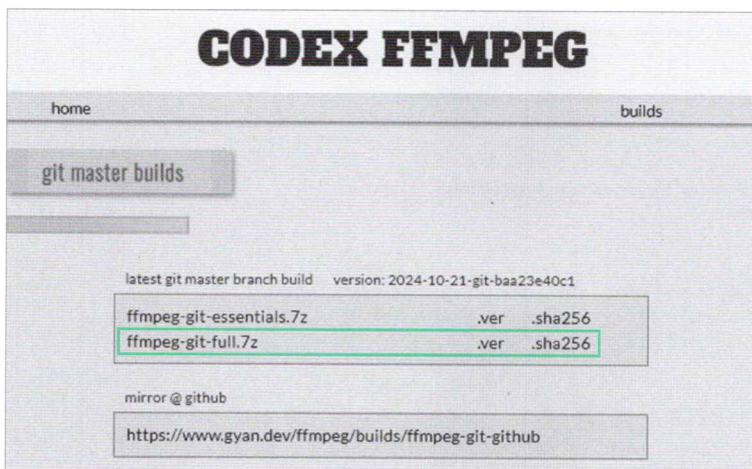
huggingface_whisper.ipynb (1)

```
%pip install --upgrade pip
%pip install --upgrade transformers datasets[audio] accelerate
```

허깅페이스의 whisper-large-v3-turbo 모델을 사용하려면 FFMPEG가 컴퓨터에 설치되어 있어야 합니다. FFMPEG는 오디오와 비디오 파일을 변환해 처리하는 오픈소스 도구입니다. 문서에서 제공한 예제 코드를 FFMPEG가 설치되지 않은 상태에서 실행하면 다음과 같은 오류가 발생합니다.

```
FileNotFoundError                                Traceback (most recent call last)
(... 생략 ...)
ValueError: ffmpeg was not found but is required to load audio files from filename
Output is truncated. View as a scrollable element or open in a text editor. Adjust
cell output settings...
```

2. FFMPEG 웹 사이트(<https://www.gyan.dev/ffmpeg/builds/>)에 접속합니다. ffmpeg-git-full.7z을 클릭해 내려받고 원하는 위치에 압축을 풀고 설치합니다.



3. FFMPEG 설치를 완료하면 새로운 셀을 만들어 다음과 같이 입력하고 실행합니다. 여기서 사용하는 경로는 자신의 컴퓨터에 설치된 FFMPEG 파일 경로를 입력합니다.

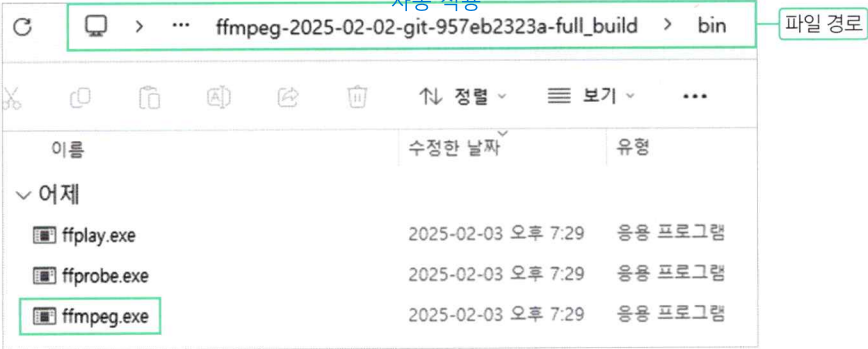
필요한 패키지 설치하기 (2) huggingface_whisper.ipynb (2)

```
import os
os.environ["PATH"] += os.pathsep + r"C:\github\gpt_agent_2025_book\ffmpeg-2025-01-22-  
full_build\bin"
```

환경변수 경로구분자
win: \ os에 따라
mac: / 구분자
(linux) 자동 적용

PATH에 등록

shared로 가져와서 이거 실행 안 함.



이제 허깅페이스의 whisper-large-v3-turbo 모델 안내 페이지에 있는 코드를 복사하여 사용해 봅시다.

Usage

Whisper large-v3-turbo is supported in Hugging Face 🤗 Transformers. To run the model, first install the Transformers library. For this example, we'll also install 🗄 Datasets to load toy audio dataset from the Hugging Face Hub, and 🚀 Accelerate to reduce the model loading time:

```
pip install --upgrade pip
pip install --upgrade transformers datasets[audio] accelerate
```

The model can be used with the pipeline class to transcribe audios of arbitrary length:

```
import torch
from transformers import AutoModelForSpeechSeq2Seq, AutoProcessor, pipeline
from datasets import load_dataset

device = "cuda:0" if torch.cuda.is_available() else "cpu"
torch_dtype = torch.float16 if torch.cuda.is_available() else torch.float32
```

복사

4. 허깅페이스에서 제공한 코드를 새로운 셀에 붙여 넣고 일부만 수정합니다.

 허깅페이스의 whisper-large-v3-turbo 코드 활용하기

 huggingface_whisper.ipynb (3)

```
import torch
from transformers import AutoModelForSpeechSeq2Seq, AutoProcessor, pipeline
# from datasets import load_dataset ①

device = "cuda:0" if torch.cuda.is_available() else "cpu"
torch_dtype = torch.float16 if torch.cuda.is_available() else torch.float32

model_id = "openai/whisper-large-v3-turbo"

model = AutoModelForSpeechSeq2Seq.from_pretrained(
    model_id, torch_dtype=torch_dtype, low_cpu_mem_usage=True, use_safetensors=True
)
model.to(device)

processor = AutoProcessor.from_pretrained(model_id)

pipe = pipeline(
    "automatic-speech-recognition",
    model=model,
    tokenizer=processor.tokenizer,
    feature_extractor=processor.feature_extractor,
    torch_dtype=torch_dtype,
    device=device,
    return_timestamps=True, ②
    chunk_length_s=10, ③
    stride_length_s=2, ③
)

# dataset = load_dataset("distil-whisper/librispeech_long", "clean", split="validation")
# sample = dataset[0]["audio"]
sample = "./audio/lxy_audio_2023_58s.mp3"

result = pipe(sample)
# print(result["text"]) ④
print(result)
```

- ❶ load_dataset에는 샘플 데이터가 들어 있습니다. 이전 예제에서 사용한 MP3 파일을 활용하면 되므로 load_dataset 관련 내용은 주석 처리하고 sample에 MP3 파일 경로를 지정합니다.
 - ❷ 오디오 파일을 처리하는 데 사용하는 클래스인 pipeline에서 return_timestamps=True로 설정하여 긴 MP3 파일을 청크 단위로 나누고 청크 시간을 각각 기록합니다. 이 설정은 화자 분리 기능을 구현할 때 유용합니다.
 - ❸ chunk_length_s를 10으로 설정해 오디오 파일을 10초씩 나눠 처리하고 각 청크가 2초씩 겹치도록 stride_length_s를 2로 설정합니다. 이렇게 하면 오디오 파일 분석의 정확도를 높일 수 있습니다.
 - ❹ 마지막으로 텍스트만 출력하게 되어 있던 부분을 result로 수정해 반환되는 값을 모두 출력하도록 수정합니다.
- ✦ 청크란 덩어리를 의미하며, 컴퓨터공학에서는 일정한 크기의 저장 공간을 말합니다.

이 코드를 실행하기 위해서 파이토치 PyTorch 라이브러리가 필요합니다. 만약 파이토치를 설치하지 않고 실행하면 다음과 같은 오류 메시지가 나옵니다.

✦ 파이토치는 페이스북의 AI 리서치 랩 FAIR에서 개발한 오픈소스 딥러닝 라이브러리입니다.

```
-----
ModuleNotFoundError                                Traceback (most recent call last)
Cell In[2], line 1
----> 1 import torch
(... 생략 ...)
ModuleNotFoundError: No module named 'torch'
```

5. 코드를 실행하기 위해 필요한 파이토치를 설치합니다. 파이토치 웹 사이트(<https://pytorch.org>)에서 사용자의 환경에 맞는 설치 명령어를 확인합니다. 여기에서는 Stable 버전과 윈도우 환경, pip 설치 방법을 선택하고 언어는 파이썬, CUDA 버전은 12.1을 선택했습니다. 명령어가 표시되면 복사합니다.

PyTorch Build	Stable (2.3.1)				Preview (Nightly)	
Your OS	Linux		Mac	Windows		
Package	Conda	Pip		LibTorch	Source	
Language	Python			C++ / Java		
Compute Platform	CUDA 11.8	CUDA 12.1	ETBBA 12.1	ROCm 6.0	CPU	
Run this Command:	<pre>pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121</pre>					

사용자 환경에 맞게 선택

복사

6. 표시된 명령어를 VS Code 터미널 창에 입력해 파이토치 라이브러리를 설치합니다.

```
(venv) > pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121
```

GPU 버전

파이토치를 설치한 후 코드를 실행합니다. 실행 결과는 딕셔너리 형태로 반환되며 그 안에는 `text`와 `chunks`가 포함됩니다. `text`에는 MP3 파일의 모든 내용이 들어 있고, `chunks`에는 시간대별로 어떤 내용이 있는지를 끊어진 리스트 형태로 정리됩니다.

dict

```
{'text': '안녕하세요. 이 강의는 GPT API로 챗봇 만들기 라는 내용을 다루는 강의입니다.'  
 (... 생략 ...)  
우리가 만들 프로그램은 이런 식으로 이제 나타나게 되고',  
 'chunks': [{'timestamp': (0.0, 6.3), 'text': '안녕하세요. 이 강의는 GPT API로 챗봇 만들기  
라는 내용을 다루는 강의입니다.'} 0.0~6.3초  
 (... 생략 ...)  
{'timestamp': (52.84, 58.0), 'text': '우리가 만들 프로그램은 이런 식으로 이제 나타나게 되고'}]}
```

7. 이 중에서 `chunks`에 저장된 값을 CSV 파일로 저장해 봅시다. 딕셔너리 `result`의 `chunks`를 `for` 문으로 순환하면서 미리 선언한 `start_end_text` 리스트에 발화 시작 시각과 종료 시각, 텍스트를 담은 뒤 리스트 형태로 추가합니다. 이 결과는 판다스 데이터프레임을 이용해 저장합니다.

◆ 데이터프레임(dataframe)이란 데이터를 행과 열로 구성된 2차원 표 형태로 정리한 데이터 구조입니다.

CSV 파일로 저장하기

huggingface_whisper.ipynb (4)

```
# chunks를 CSV 파일로 저장  
start_end_text = []  
  
for chunk in result["chunks"]:  
    start = chunk["timestamp"][0]  
    end = chunk["timestamp"][1]  
    text = chunk["text"]  
    start_end_text.append([start, end, text])  
  
import pandas as pd  
df = pd.DataFrame(start_end_text, columns=["start", "end", "text"])  
df.to_csv("lsy_audio_2023_58.csv", index=False, sep="|")  
display(df)
```

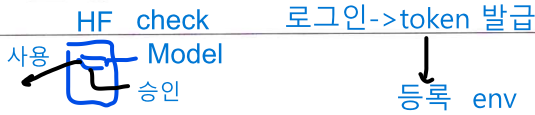
이 셀을 실행하면 다음과 같이 데이터프레임으로 정리된 내용이 출력됩니다.

	start	end	text
0	0.00	6.30	안녕하세요. 이 강의는 GPT API로 챗봇 만들기 라는 내용을 다루는 강의입니다.
1	7.18	10.00	GPT API에 대해서 생소하신 분들도 있을텐데
2	11.00	17.00	우리가 잘 알고 있는 ChatGPT, ChatGPT 기능을 이용해서
3	17.00	21.00	우리가 원하는 프로그램을 어떻게 만드는데 대해서 이야기할 거예요.
4	21.00	24.00	그래서 이런 강의들이 사실 많이 있습니다.
5	24.00	27.48	그래서 여러 가지들이 있는데 이 강의 특징이라고 한다면
6	27.48	29.58	GPT로 명확한 미션을 달성하는
7	29.58	31.66	챗봇 프로그램을 만드는데 사실
8	31.66	34.32	쉽지는 않은데 이걸 어떻게 해서
9	34.32	36.40	구현을 하는지 그리고 그게 왜 필요한지에 대해서
10	36.40	37.36	좀 이야기를 할 거고요.
11	38.00	40.00	그 예제로 예제는 여러가지가 될 수 있는데
12	40.00	42.00	여기서 예제로 하는 것은
13	42.00	44.20	음악 플레이리스트 동영상
14	44.20	47.10	자동으로 대화를 통해서 생성하는 프로그램을 만드는 것을
15	47.10	48.46	다루려고 합니다.
16	49.84	51.96	그래서 프로그램이 실행되는 모습을 한번 보여드릴게요.
17	52.84	58.00	우리가 만들 프로그램은 이런 식으로 이제 나타나게 되고

CSV로 저장된 파일도 열어 보면 다음과 같이 잘 정리되어 있습니다.

CSV 저장 결과		lsy_audio_2023_58.csv
start	end	text
0.0	6.3	안녕하세요. 이 강의는 GPT API로 챗봇 만들기 라는 내용을 다루는 강의입니다.
7.18	10.0	GPT API에 대해서 생소하신 분들도 있을텐데
11.0	17.0	우리가 잘 알고 있는 ChatGPT, ChatGPT 기능을 이용해서
(... 생략 ...)		
49.84	51.96	그래서 프로그램이 실행되는 모습을 한번 보여드릴게요.
52.84	58.0	우리가 만들 프로그램은 이런 식으로 이제 나타나게 되고

05-3 문장과 화자 구분하기



설치 시 버전지정 안 하면
최신버전으로 설치(연관된 파일들도
최신으로 올라갈 수 있음.)

프로젝트 전에 버전 맞추고 시작하기
(constraint.txt 맞추기?)

위스퍼 API는 음성을 잘 받아쓰지만 누가 어떤 말을 했는지 구분해 주지는 않습니다. 회의처럼 누가 말했는지가 중요한 경우에는 발언한 사람의 정보를 포함해서 회의록을 만들면 더욱 좋겠죠. 이번 절에서는 여러 사람의 대화에서 화자를 구분하고 누가 어떤 말을 했는지 분석해 보겠습니다.

Do it! 실습 화자 분리 모델로 시간대별 화자 구분하기

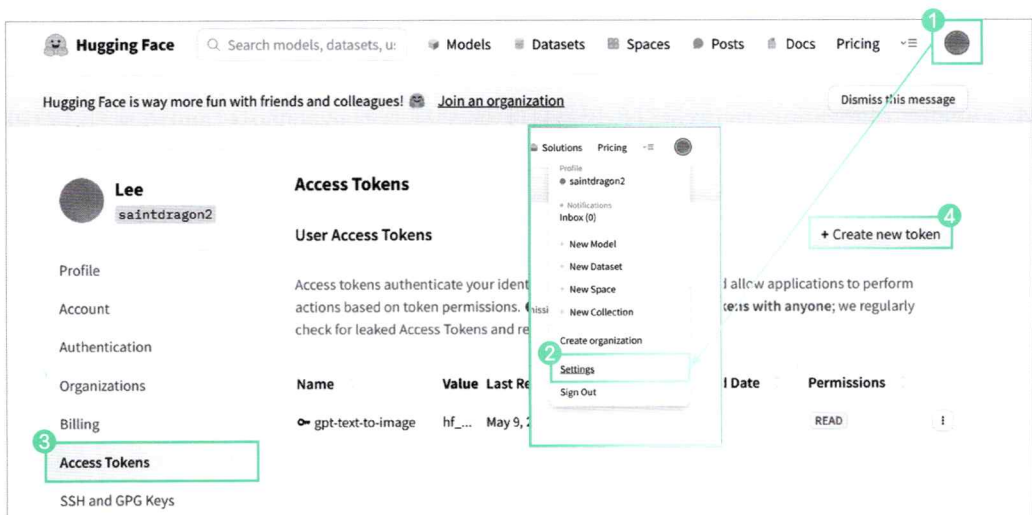
결과 파일: sec03/speaker_diarization.ipynb

화자를 구분하는 기능은 `pyannote.audio`를 이용해 개발할 수 있습니다. `pyannote.audio`는 화자를 분리하는 오픈소스 툴킷으로 파이토치 머신러닝 프레임워크에서 동작합니다. 이 실습에서는 이론보다 기능 구현에 집중해 보겠습니다.

◆ `pyannote.audio`가 동작하는 방식이 궁금하다면 문서 <https://pypi.org/project/pyannote.audio/>를 참고하세요.

허깅페이스에서 화자 분리 모델 내려받고 사용 준비하기

1. 화자 분리 모델을 사용하려면 토큰을 발급받아야 합니다. 허깅페이스 웹 사이트(<https://huggingface.co>)에 로그인한 뒤 프로필 로고를 클릭하고 [Settings]를 클릭합니다. 이동한 페이지에서 [Access Tokens]를 선택하고 [+Create new token]을 선택해 토큰을 발급받습니다.



- pyannote : 화자 분리 => 음성(녹음 파일)로 말하는 사람 분리
1. 분할(Segmentation)
-> 오디오 쪼개기: 10초 단위, 슬라이딩(stride와 비슷)
 2. 목소리 지문 추출(Embedding: data를 vector로 변경(의미 부여))
음색, 톤, 성대 구조적 특징 등 추출
 3. 묶기(Clustering)

빨, 파는 비슷한 데이터.
같은 사람의 목소리라고 할 수 O

초는 비슷X.
다른 사람 목소리라고 할 수 O

2. pyannote.audio는 허깅페이스에서 내려받아 사용할 수 있습니다. 허깅페이스 웹 사이트에서 'speaker-diarization'을 검색하고 pyannote의 speaker-diarization-3.1 모델 페이지를 찾습니다.



upload하려면 write 필요

3. 모델을 사용하려면 소속 기관과 웹 사이트 정보를 입력해야 합니다. 여기에서는 [Company/university]에 'PERSONAL'을, [Website]에 'I have no website'를 입력했습니다. 그리고 [Agree and access repository]를 클릭합니다.

4. speaker_diarization.ipynb 파일을 만듭니다. 첫 번째 셀에 pyannote.audio 패키지와 이를 사용하는 데 필요한 numpy 패키지를 설치합니다. 2025년 4월 기준으로 pyannote.audio는 numpy 1.x 버전에서 작동하도록 되어 있으므로 numpy는 1.26 버전으로 설치합니다.

```

pyannote.audio와 numpy 설치하기
speaker_diarization.ipynb (1)

%pip install pyannote.audio
%pip install numpy==1.26

```

모델 상세 페이지인 Usage 영역의 예제 코드를 참고해서 코드를 작성해 보겠습니다. 해당 코드는 pyannote.audio 라이브러리를 임포트하고 Pipeline.from_pretrained 메서드를 사용하여 허깅페이스에 호스팅된 pyannote/speaker-diarization-3.1 모델을 불러옵니다.

Usage

```
# instantiate the pipeline
from pyannote.audio import Pipeline
pipeline = Pipeline.from_pretrained(
    "pyannote/speaker-diarization-3.1",
    use_auth_token="HUGGINGFACE_ACCESS_TOKEN_GOES_HERE")

# run the pipeline on an audio file
diarization = pipeline("audio.wav")

# dump the diarization output to disk using RTTM format
with open("audio.rttm", "w") as rttm:
    diarization.write_rttm(rttm)
```

5. 다음과 같이 코드를 작성합니다. 코드에서 use_auth_token은 계정에서 생성한 토큰으로 수정합니다. 이 토큰은 모델에 접근하는 인증 수단으로 사용합니다. 또한 CUDA를 지원하는 컴퓨터에서는 이를 활용해 작업 속도를 높일 수 있도록 관련 코드를 추가합니다. 이렇게 pipeline을 선언하면 준비 끝입니다.

◆ CUDA를 지원하는 GPU가 있다면 CUDA를 이용하도록 했습니다. CUDA를 지원하지 않는 컴퓨터에서도 실행할 수 있지만 처리 속도가 느릴 수 있습니다.

pyannote.audio 사용하기

speaker_diarization.ipynb (2)

```
# instantiate the pipeline
from pyannote.audio import Pipeline
pipeline = Pipeline.from_pretrained(
    "pyannote/speaker-diarization-3.1",
    use_auth_token="HUGGINGFACE_ACCESS_TOKEN_GOES_HERE" — 토큰 입력
)

# CUDA를 사용할 수 있다면 CUDA를 사용하도록 설정
if torch.cuda.is_available():
    pipeline.to(torch.device("cuda"))
    print('cuda is available')
else:
    print('cuda is not available')
```

이 상태에서 코드를 실행하면 다음과 같은 오류 메시지가 나타날 수 있습니다.

```
Could not download 'pyannote/segmentation-3.0' model.  
It might be because the model is private or gated so make  
sure to authenticate. Visit https://hf.co/settings/tokens to  
create your access token and retry with:  
(... 생략 ...)  
AttributeError: 'NoneType' object has no attribute 'eval'
```

6. pyannote/speaker-diarization-3.1 모델을 사용하려면 pyannote/segmentation-3.0의 사용 권한을 획득해야 합니다. 방법은 pyannote/speaker-diarization-3.1 상세 페이지의 Requirements 부분에 안내되어 있습니다. 여기서 [pyannote/segmentation-3.0] 링크를 클릭합니다.

Requirements

1. Install `pyannote.audio` 3.1 with `pip install pyannote.audio`
2. Accept `pyannote/segmentation-3.0` user conditions
3. Accept `pyannote/speaker-diarization-3.1` user conditions
4. Create access token at hf.co/settings/tokens.

7. pyannote/segmentation-3.0 페이지에서 사용 권한을 확보하기 위해 소속 기관과 웹사이트를 적고 [Agree and access repository]를 클릭합니다. 여기에서는 각각 'PERSONAL'과 'I have no website'로 입력했습니다.

By agreeing you accept to share your contact information (email and username) with the repository authors.

Company/university
PERSONAL
Website
I have no website
☒ Agree and access repository

음성 파일에서 화자 분리하기

이제 준비가 끝났으니 음성 파일에서 화자를 분리해 보겠습니다. 화자 분리 테스트를 위해 화자가 최소 2명 이상 등장하는 음성 파일을 사용합니다. 저는 전기 기타를 배울 때 저렴한 기타와 비싼 기타 중에서 어떤 것으로 시작하는 것이 좋은지 AI와 토론한 MP3 파일을 사용하겠습니다.

예제와 같은 파일로 실험하고 싶다면 깃허브에서 음성 파일을 내려받으세요(https://github.com/saindragon2/gpt_agent_2025_easyspub/blob/main/chap05/audio/싼기타_비싼기타.mp3)

앞에서 선언한 pipeline을 이용해 MP3 파일을 화자 분리 객체 diarization로 만들고 그 결과를 RTTM 파일로 저장합니다. 이때 MP3 파일과 저장할 RTTM 파일의 폴더 경로를 정확히 입력하세요.

화자 분리하고 RTTM 파일로 저장하기 speaker_diarization.ipynb (3)

```
# run the pipeline on an audio file
diarization = pipeline("./audio/싼기타_비싼기타.mp3")
# dump the diarization output to disk using RTTM format
with open("./audio/싼기타_비싼기타.rttm", "w", encoding='utf-8') as rttm:
    diarization.write_rttm(rttm)
```

코드를 실행하면 다음과 같은 결과 파일이 생성됩니다. 발화 시작 시각, 지속 시각, 그리고 화자가 SPEAKER_00, SPEAKER_01로 구분되어 있습니다. 예를 들어 SPEAKER_01의 발언은 0.093초부터 5.805초 동안, 그리고 7.405초부터 3.983초 동안 이어집니다. 이런 식으로 SPEAKER_01의 발언이 계속되다가 32.414초부터 SPEAKER_00의 발언이 나옵니다.

컴퓨터 환경에 따라 경고 메시지가 출력될 수 있지만 큰 문제는 아닙니다. 대부분 라이브러리 간의 버전 차이로 발생하는 메시지이므로 결과가 잘 출력되면 무시해도 괜찮습니다.

화자 분리 실행 결과		start duration		화자		■ 싼기타_비싼기타.rttm
SPEAKER	싼기타_비싼기타	1	0.993 5.805	<NA> <NA>	SPEAKER_01	<NA> <NA>
SPEAKER	싼기타_비싼기타	1	7.405 3.983	<NA> <NA>	SPEAKER_01	<NA> <NA>
SPEAKER	싼기타_비싼기타	1	11.759 4.927	<NA> <NA>	SPEAKER_01	<NA> <NA>
SPEAKER	싼기타_비싼기타	1	17.210 10.665	<NA> <NA>	SPEAKER_01	<NA> <NA>
SPEAKER	싼기타_비싼기타	1	28.668 1.536	<NA> <NA>	SPEAKER_01	<NA> <NA>
SPEAKER	싼기타_비싼기타	1	32.414 0.759	<NA> <NA>	SPEAKER_00	<NA> <NA>
SPEAKER	싼기타_비싼기타	1	33.545 3.561	<NA> <NA>	SPEAKER_00	<NA> <NA>
SPEAKER	싼기타_비싼기타	1	37.628 3.763	<NA> <NA>	SPEAKER_00	<NA> <NA>
(... 생략 ...)						

판다스를 활용해 데이터프레임 형태로 저장하기

화자 분리는 잘 되지만 한 화자의 발언이 여러 행에 나누어 출력되는 문제가 있습니다. 같은 화자가 계속 이야기하는 경우에는 하나로 합쳐 보겠습니다. 판다스 `pandas`를 이용하면 데이터프레임 형태의 데이터를 쉽게 조작할 수 있습니다. 원하는 형태로 데이터를 정리한 뒤 CSV 파일로 저장해 보겠습니다.

1. 앞에서 생성한 `썸기타_비썸기타.rttm` 파일을 보면 빈칸으로 구분된 10개의 정보 항목이 나열되어 있습니다. 따라서 판다스의 `.read_csv`를 사용해 RTTM 파일의 데이터를 빈칸을 기준으로 나누어서 읽습니다. 이때 10개 정보 항목의 이름을 `names` 변수에서 정의한 대로 지정해서 읽습니다.

RTTM을 CSV로 변환하기

 `speaker_diarization.ipynb (4)`

```
import pandas as pd
rttm_path = "./audio/썸기타_비썸기타.rttm"

df_rttm = pd.read_csv(
    rttm_path,      # rttm 파일 경로
    sep=' ',        # 구분자는 띄어쓰기
    header=None,    # 헤더는 없음
    names=['type', 'file', 'chnl', 'start', 'duration', 'C1', 'C2', 'speaker_id', 'C3', 'C4']
)

display(df_rttm)
```

이 코드를 실행하면 다음처럼 빈칸을 기준으로 데이터가 나누어진 데이터프레임이 출력됩니다.

	type	file	chnl	start	duration	C1	C2	speaker_id	C3	C4
0	SPEAKER	썸기타_비썸기타	1	0.993	5.805	NaN	NaN	SPEAKER_01	NaN	NaN
1	SPEAKER	썸기타_비썸기타	1	7.405	3.983	NaN	NaN	SPEAKER_01	NaN	NaN
2	SPEAKER	썸기타_비썸기타	1	11.759	4.927	NaN	NaN	SPEAKER_01	NaN	NaN
3	SPEAKER	썸기타_비썸기타	1	17.210	10.665	NaN	NaN	SPEAKER_01	NaN	NaN
4	SPEAKER	썸기타_비썸기타	1	28.668	1.536	NaN	NaN	SPEAKER_01	NaN	NaN
...	...	...	...	...	...	...	...	...	...	...
83	SPEAKER	썸기타_비썸기타	1	414.481	2.970	NaN	NaN	SPEAKER_00	NaN	NaN
84	SPEAKER	썸기타_비썸기타	1	417.755	3.476	NaN	NaN	SPEAKER_01	NaN	NaN
85	SPEAKER	썸기타_비썸기타	1	423.644	0.776	NaN	NaN	SPEAKER_00	NaN	NaN
86	SPEAKER	썸기타_비썸기타	1	424.741	3.527	NaN	NaN	SPEAKER_00	NaN	NaN
87	SPEAKER	썸기타_비썸기타	1	428.504	0.844	NaN	NaN	SPEAKER_00	NaN	NaN

88 rows × 10 columns

2. 출력된 내용에는 발언이 시작된 시간인 **start**와 발언이 지속된 시간인 **duration**이 있으므로 이를 활용해 각 발언이 언제 끝났는지를 계산할 수 있습니다. 분석 결과의 **start**에 **duration**을 더해서 발언이 끝난 시간인 **end**를 계산하고 이를 결과에 추가합니다.

발언이 끝난 시간 추가하기

speaker_diarization.ipynb (5)

```
# start + duration을 end로 변환
df_rttm['end'] = df_rttm['start'] + df_rttm['duration']

display(df_rttm)
```

코드를 실행해 보면 출력 결과에 **end**가 추가됩니다.

	type	file	chnl	start	duration	C1	C2	speaker_id	C3	C4	end
0	SPEAKER	싼기타_비싼기타	1	0.993	5.805	NaN	NaN	SPEAKER_01	NaN	NaN	6.798
1	SPEAKER	싼기타_비싼기타	1	7.405	3.983	NaN	NaN	SPEAKER_01	NaN	NaN	11.388
2	SPEAKER	싼기타_비싼기타	1	11.759	4.927	NaN	NaN	SPEAKER_01	NaN	NaN	16.686
3	SPEAKER	싼기타_비싼기타	1	17.210	10.665	NaN	NaN	SPEAKER_01	NaN	NaN	27.875
4	SPEAKER	싼기타_비싼기타	1	28.668	1.536	NaN	NaN	SPEAKER_01	NaN	NaN	30.204
...	...	...	...	...	...	...	...	...	...	...	...
83	SPEAKER	싼기타_비싼기타	1	414.481	2.970	NaN	NaN	SPEAKER_00	NaN	NaN	417.451
84	SPEAKER	싼기타_비싼기타	1	417.755	3.476	NaN	NaN	SPEAKER_01	NaN	NaN	421.231
85	SPEAKER	싼기타_비싼기타	1	423.644	0.776	NaN	NaN	SPEAKER_00	NaN	NaN	424.420
86	SPEAKER	싼기타_비싼기타	1	424.741	3.527	NaN	NaN	SPEAKER_00	NaN	NaN	428.268
87	SPEAKER	싼기타_비싼기타	1	428.504	0.844	NaN	NaN	SPEAKER_00	NaN	NaN	429.348

88 rows × 11 columns

화자를 구분하고 발언 순서를 기록하기 위해 화자가 바뀔 때마다 발언에 번호를 부여하겠습니다. 예를 들어 **SPEAKER_00**이 이야기하는 동안에는 1, 다음으로 **SPEAKER_01**이 이야기하는 동안에는 2, 다시 **SPEAKER_00**이 이야기를 시작하면 3이 되도록 설정하겠습니다.

3. **발언 번호인 number**를 **담을 열**을 만들고 **None**으로 초기화합니다. 그리고 번호를 매기기 시작하는 기준점을 만들기 위해 첫 번째 행에만 **number**를 **0**으로 지정합니다.

연속된 발화를 기록하기 위해 number 변수 추가하기

speaker_diarization.ipynb (6)

```
df_rttm["number"] = None # number 열 만들고 None으로 초기화
df_rttm.at[0, "number"] = 0 첫번째 음성에 0
# 행 열

display(df_rttm)
```

이 셀을 실행하면 다음과 같이 첫 번째 행(0번 행)만 number가 0으로 되어 있고 나머지는 None인 상태로 남아 있습니다.

	type	file	chnl	start	duration	C1	C2	speaker_id	C3	C4	end	number
0	SPEAKER	싼기타_비싼기타	1	0.993	5.805	NaN	NaN	SPEAKER_01	NaN	NaN	6.798	0
1	SPEAKER	싼기타_비싼기타	1	7.405	3.983	NaN	NaN	SPEAKER_01	NaN	NaN	11.388	None
2	SPEAKER	싼기타_비싼기타	1	11.759	4.927	NaN	NaN	SPEAKER_01	NaN	NaN	16.686	None
3	SPEAKER	싼기타_비싼기타	1	17.210	10.665	NaN	NaN	SPEAKER_01	NaN	NaN	27.875	None
4	SPEAKER	싼기타_비싼기타	1	28.668	1.536	NaN	NaN	SPEAKER_01	NaN	NaN	30.204	None
...	...	...	...	...	...	...	...	...	...	...	...	...
83	SPEAKER	싼기타_비싼기타	1	414.481	2.970	NaN	NaN	SPEAKER_00	NaN	NaN	417.451	None

4. 두 번째 행(i = 1)부터 시작해서 이전 행(i - 1)의 speaker_id가 같으면 그 행의 number를 그대로 가져오고, 다르면 number에 1을 더해 새로운 번호를 붙입니다. 즉, 이전 화자와 같은 번호를 그대로 사용하고 다르면 번호를 1씩 증가시킵니다.

화자 번호 매기기

speaker_diarization.ipynb (7)

```
for i in range(1, len(df_rttm)):  
    if df_rttm.at[i, "speaker_id"] != df_rttm.at[i-1, "speaker_id"]:  
        df_rttm.at[i, "number"] = df_rttm.at[i-1, "number"] + 1  
    else:  
        df_rttm.at[i, "number"] = df_rttm.at[i-1, "number"]  
  
display(df_rttm.head(10))
```

실행해 보면 한 화자가 말하는 동안 number가 같은 번호로 이어지고 화자가 바뀌면 번호가 증가합니다.

	type	file	chnl	start	duration	C1	C2	speaker_id	C3	C4	end	number
0	SPEAKER	싼기타_비싼기타	1	0.993	5.805	NaN	NaN	SPEAKER_01	NaN	NaN	6.798	0
1	SPEAKER	싼기타_비싼기타	1	7.405	3.983	NaN	NaN	SPEAKER_01	NaN	NaN	11.388	0
2	SPEAKER	싼기타_비싼기타	1	11.759	4.927	NaN	NaN	SPEAKER_01	NaN	NaN	16.686	0
3	SPEAKER	싼기타_비싼기타	1	17.210	10.665	NaN	NaN	SPEAKER_01	NaN	NaN	27.875	0
4	SPEAKER	싼기타_비싼기타	1	28.668	1.536	NaN	NaN	SPEAKER_01	NaN	NaN	30.204	0
5	SPEAKER	싼기타_비싼기타	1	32.414	0.759	NaN	NaN	SPEAKER_00	NaN	NaN	33.173	1
6	SPEAKER	싼기타_비싼기타	1	33.545	3.561	NaN	NaN	SPEAKER_00	NaN	NaN	37.106	1
7	SPEAKER	싼기타_비싼기타	1	37.628	3.763	NaN	NaN	SPEAKER_00	NaN	NaN	41.391	1
8	SPEAKER	싼기타_비싼기타	1	41.611	1.097	NaN	NaN	SPEAKER_00	NaN	NaN	42.708	1
9	SPEAKER	싼기타_비싼기타	1	41.645	0.810	NaN	NaN	SPEAKER_01	NaN	NaN	42.455	2

5. 이제 `number`가 같은 행들을 하나로 묶고 `start`는 최솟값, `end`는 최댓값으로 설정하여 한 화자가 말한 시간 범위를 정리합니다. 이렇게 처리한 결과는 `df_rttm_grouped`에 담습니다.

같은 화자끼리 묶어서 정리하기

speaker_diarization.ipynb (8)

```
df_rttm_grouped = df_rttm.groupby("number").agg(
    start=pd.NamedAgg(column='start', aggfunc='min'),
    end=pd.NamedAgg(column='end', aggfunc='max'),
    speaker_id=pd.NamedAgg(column='speaker_id', aggfunc='first')
)

display(df_rttm_grouped)
```

셀을 실행한 결과, `speaker_id`가 같은 경우에는 해당 화자의 발언들이 하나의 행으로 묶여서 정리되었습니다. 화자별로 `start`는 최솟값, `end`는 최댓값으로 설정되어 발언 시간대가 잘 합쳐졌습니다. 지금 결과는 판다스의 `.groupby`를 이용해서 `number`가 인덱스로 지정되었습니다. 최종 결과를 출력할 때는 일반 열로 변경해야 처리하기가 더 쉬우므로 인덱스를 삭제하는 것이 좋습니다.

	start	end	speaker_id
number			
0	0.993	30.204	SPEAKER_01
1	32.414	42.708	SPEAKER_00
2	41.645	44.024	SPEAKER_01
3	45.813	67.109	SPEAKER_00
4	67.227	82.786	SPEAKER_01
5	84.659	102.564	SPEAKER_00

6. 화자별 발화 시간을 구하고 인덱스를 제거하겠습니다. `end`에서 `start`를 빼면 `duration`을 구할 수 있습니다. 그리고 `.reset_index`로 인덱스를 제거합니다.

발화 시간 추가하고 인덱스 제거하기

speaker_diarization.ipynb (9)

```
df_rttm_grouped["duration"] = df_rttm_grouped["end"] - df_rttm_grouped["start"]
df_rttm_grouped = df_rttm_grouped.reset_index(drop=True)
display(df_rttm_grouped)
```

이 셀을 실행하면 duration 열이 추가되고 인덱스였던 number가 삭제된 결과가 출력됩니다.

	start	end	speaker_id	duration
0	0.993	30.204	SPEAKER_01	29.211
1	32.414	42.708	SPEAKER_00	10.294
2	41.645	44.024	SPEAKER_01	2.379
3	45.813	67.109	SPEAKER_00	21.296
4	67.227	82.786	SPEAKER_01	15.559
5	84.659	102.564	SPEAKER_00	17.905

7. 이 내용을 CSV 파일로 저장합니다. 인덱스(0, 1, 2, ...)는 저장하지 않고 파일명은 '싼기타_비싼기타_rttm.csv'로 지정합니다. 파일 경로는 원하는 대로 선택하세요.

화자 분리 결과를 CSV 파일로 저장하기

speaker_diarization.ipynb (10)

```
df_rttm_grouped.to_csv(
    "./audio/싼기타_비싼기타_rttm.csv",
    sep=',',
    index=False
)
```

셀을 실행하고 생성된 파일을 열어 보면 결과가 잘 저장되어 있습니다.

CSV 파일 저장 결과

싼기타_비싼기타_rttm.csv

```
start,end,speaker_id,duration
0.993,30.204,SPEAKER_01,29.211000000000002
32.414,42.708,SPEAKER_00,10.293999999999997
41.645,44.024,SPEAKER_01,2.378999999999998
45.813,67.109000000000001,SPEAKER_00,21.296000000000006
(... 생략 ...)
```

Do it! 실습 판다스로 문장 분석하고 화자 매칭하기

결과 파일: sec03/whisper_stt.py

앞에서 만든 RTTM 파일에 있는 화자 데이터와 위스퍼 API를 이용해 음성을 받아쓴 CSV 파일의 데이터를 분석하면 누가 어떤 문장을 말했는지 알 수 있습니다. 이를 분석하기 위해 같은 음성 파일을 기반으로 한 CSV 파일과 RTTM 파일을 준비하겠습니다. 여기서는 앞선 실습에서 사용한 '싼기타_비싼기타.mp3' 음성 파일을 활용하겠습니다.

모듈 -> 다른 py에서 import

재사용 목적, 함수로 묶어서 한 번에 import 하기 위해?

로컬에 설치한 위스퍼 모델을 활용해 받아쓰기 함수 만들기

1. whisper_stt.py 파일을 만듭니다. MP3 파일의 음성을 텍스트로 변환하고 그 결과를 판다스 데이터프레임으로 정리해서 CSV 파일로 저장하는 코드를 작성하겠습니다. 이 중에서 받아쓰기하는 기능을 whisper_stt 함수로 만들고, 판다스 데이터프레임 형태로 저장하는 기능은 whisper_to_dataframe 함수로 만들어 보겠습니다. 05-2절에서 만든 huggingface_whisper.ipynb 파일의 코드를 활용해 받아쓰기하는 파이썬 함수 whisper_stt를 만듭니다. 기존 예제 코드에서 주요 변경 사항은 경로를 함수의 매개변수로 수정하는 것입니다.

받아쓰기하는 함수 whisper_stt 만들기 (1)

whisper_stt.py

```
import os
import torch
import pandas as pd
from transformers import AutoModelForSpeechSeq2Seq, AutoProcessor, pipeline

os.environ["PATH"] += os.pathsep + r"C:\github\gpt_agent_2025_book\ffmpeg-2025-01-22-
full_build\bin" # 자신이 설치한 위치로 경로 수정

def whisper_stt(
    audio_file_path: str,
    output_file_path: str = "./output.csv"
):
    device = "cuda:0" if torch.cuda.is_available() else "cpu"
    torch_dtype = torch.float16 if torch.cuda.is_available() else torch.float32
    model_id = "openai/whisper-large-v3-turbo"

    model = AutoModelForSpeechSeq2Seq.from_pretrained(
        model_id, torch_dtype=torch_dtype,
        low_cpu_mem_usage=True,
        use_safetensors=True
    )
    model.to(device)

    processor = AutoProcessor.from_pretrained(model_id)

    pipe = pipeline(
        "automatic-speech-recognition",
        model=model,
        tokenizer=processor.tokenizer,
        feature_extractor=processor.feature_extractor,
        torch_dtype=torch_dtype,
        device=device,
```

```

    return_timestamps=True, # 청크별로 타임스탬프를 반환
    chunk_length_s=10,     # 입력 오디오를 10초씩 나누기
    stride_length_s=2,     # 청크가 2초씩 겹치도록 나누기
)

result = pipe(audio_file_path)
df = whisper_to_dataframe(result, output_file_path)

return result, df

```

3 다음 코드에서 선언

- 1 필요한 라이브러리들을 05-2절에서 만든 코드에서 가져와 임포트합니다.
- 2 `whisper_stt` 함수의 매개변수는 `audio_file_path`와 `output_file_path`입니다. 음성 파일 경로를 `audio_file_path`에 넣으면 결과가 `output_file_path`에 CSV 형식으로 저장됩니다.
- 3 위스퍼 모델을 이용해 얻은 결과를 CSV 파일로 저장하기 위해 `whisper_to_dataframe` 함수를 사용합니다. `whisper_to_dataframe` 함수는 이어지는 코드에서 선언합니다.

2. `whisper_to_dataframe` 함수는 05-2절에서 만든 코드와 같습니다. `result` 매개변수로 받은 결과를 판다스 데이터프레임으로 변환하고 `output_file_path`로 받은 경로에 CSV 파일로 저장한 후 데이터프레임을 반환합니다.

받아쓰기하는 함수 `whisper_stt` 만들기 (2)

whisper_stt.py

```

(... 생략 ...)
def whisper_to_dataframe(result, output_file_path):
    start_end_text = []

    for chunk in result["chunks"]:
        start = chunk["timestamp"][0]
        end = chunk["timestamp"][1]
        text = chunk["text"].strip()
        start_end_text.append([start, end, text])
    df = pd.DataFrame(start_end_text, columns=["start", "end", "text"])
    df.to_csv(output_file_path, index=False, sep="|")

    return df

```

3. 이제 다음처럼 메인 영역을 선언하고 `whisper_stt` 함수를 실행한 후 결과를 출력합니다. 오디오 파일 경로와 결과를 저장할 CSV 파일 경로는 여러분의 환경에 맞게 설정하세요.

```

받아쓰기하는 함수 whisper_stt 만들기 (3) ■ whisper_stt.py

(... 생략 ...)
if __name__ == "__main__":
    result, df = whisper_stt(
        "./chap05/audio/싼기타_비싼기타.mp3",
        "./chap05/audio/싼기타_비싼기타.csv",
    )

    print(df)

```

완성한 코드를 실행하면 다음처럼 음성 파일을 텍스트로 받아써서 시간대별로 나눈 결과가 터미널 창에 출력되고 CSV 파일로도 저장됩니다. 몇 가지 잘못 받아 적은 내용이 보이지만 내용 파악은 할 수 있을 것 같습니다.

```

문제 20 출력 디버그 콘솔 터미널 포트 JUPYTER

      start      end      text
0      1.00      4.00      지금부터 저랑 역할극을 합시다.
1      4.00      7.00      역할극을 스탠드인 코미디 스타일로 할 건데
2      7.00      10.08     토론을 하면서 자연스럽게 대화하는 형식으로 하면서
3      10.08     11.22     코미디를 진행해봅시다.
4      11.82     16.04     그래서 좀 재밌고 자연스럽게 유머러스하게 저랑 대화를 하시면 돼요.
...      ...      ...
110     398.00    406.50     아니요, 전혀 화나지 않았어요. 편하게 이야기 나누는 중이니까 걱정하세요.
111     407.24    408.14     화난 것 같은데?
112     414.00    414.54     네, 정말 괜찮습니다. 즐겁게 대화 나누고 있었어요.
113     417.00    423.00     계속해서 이하고 나누고 싶으시면 편하게 말씀해주세요. 아니요. 화났었는데 굳이 더 ...
114     423.00    428.96     알겠습니다. 언제든 다시 이야기 나누고 싶으실 때 편하게 말씀해 주세요. 감사합니다.

```

```

CSV 파일 저장 결과 ■ 싼기타_비싼기타.csv

start|end|text
1.0|4.0|지금부터 저랑 역할극을 합시다.
4.0|7.0|역할극을 스탠드인 코미디 스타일로 할 건데
7.0|10.08|토론을 하면서 자연스럽게 대화하는 형식으로 하면서
10.08|11.22|코미디를 진행해봅시다.
11.82|16.04|그래서 좀 재밌고 자연스럽게 유머러스하게 저랑 대화를 하시면 돼요.
(... 생략 ...)

```

시간대별로 화자를 구분하는 함수 만들기

05-3절에서 화자 분리 연습을 할 때 사용했던 speaker_diarization.ipynb 파일 내용을 함수로 정리해서 whisper_stt.py 파일에 추가하겠습니다.

1. 앞서 만들었던 화자 분리 기능을 speaker_diarization 함수로 만듭니다.

시간대별로 화자를 구분하는 함수 speaker_diarization

whisper_stt.py

```
import os
import torch
import pandas as pd
from transformers import AutoModelForSpeechSeq2Seq, AutoProcessor, pipeline
from pyannote.audio import Pipeline

(... 생략 ...)
def whisper_to_dataframe(result, output_file_path):
    (... 생략 ...)
    return df

def speaker_diarization(
    audio_file_path: str,
    output_rttm_file_path: str,
    output_csv_file_path: str
):
    pipeline = Pipeline.from_pretrained(
        "pyannote/speaker-diarization-3.1",
        use_auth_token="HUGGINGFACE_ACCESS_TOKEN_GOES_HERE"
    )
    # CUDA를 사용할 수 있다면 CUDA를 사용하도록 설정
    if torch.cuda.is_available():
        pipeline.to(torch.device("cuda"))
        print('cuda is available')
    else:
        print('cuda is not available')
    diarization_pipeline = pipeline(audio_file_path)

    # dump the diarization output to disk using RTTM format
    with open(output_rttm_file_path, "w", encoding='utf-8') as rttm:
        diarization_pipeline.write_rttm(rttm)

    # 판다스 데이터프레임으로 변환
    df_rttm = pd.read_csv(
```

```

    output_rttm_file_path, # rttm 파일 경로
    sep=',',              # 구분자는 띄어쓰기
    header=None,          # 헤더는 없음
    names=['type', 'file', 'chnl', 'start', 'duration', 'C1', 'C2', 'speaker_id',
'C3', 'C4']
)

df_rttm["end"] = df_rttm["start"] + df_rttm["duration"]

# speaker_id를 기반으로 화자별로 구간 나누기
df_rttm["number"] = None
df_rttm.at[0, "number"] = 0

for i in range(1, len(df_rttm)):
    if df_rttm.at[i, "speaker_id"] != df_rttm.at[i-1, "speaker_id"]:
        df_rttm.at[i, "number"] = df_rttm.at[i-1, "number"] + 1
    else:
        df_rttm.at[i, "number"] = df_rttm.at[i-1, "number"]

df_rttm_grouped = df_rttm.groupby("number").agg(
    start=pd.NamedAgg(column='start', aggfunc='min'),
    end=pd.NamedAgg(column='end', aggfunc='max'),
    speaker_id=pd.NamedAgg(column='speaker_id', aggfunc='first')
)

df_rttm_grouped["duration"] = df_rttm_grouped["end"] - df_rttm_grouped["start"]

df_rttm_grouped.to_csv(
    output_csv_file_path,
    index=False, # 인덱스는 저장하지 않음
    encoding='utf-8'
)
return df_rttm_grouped
(... 생략 ...)

```

- ❶ 매개변수로 audio_file_path와 output_rttm_file_path, output_csv_file_path로 입출력할 파일 경로를 추가합니다.
- ❷ whisper_stt.py 파일 내에 여러 파이프라인이 있으므로 가독성을 높이기 위해 기존 코드의 pipeline이라 변수명을 diarization_pipeline으로 수정합니다. 이 함수를 실행하면 MP3 파일의 음성을 분석하여 화자를 시간대별로 구분하고 이를 CSV 파일로 저장합니다.

2. 파이썬 코드의 메인 부분도 이에 맞게 수정합니다. 인풋과 아웃풋의 파일 경로는 여러 함수에서 사용하므로 이를 별도의 변수로 선언합니다. 현재는 `speaker_diarization` 함수만 테스트하면 되므로 시간을 절약하기 위해 `whisper_stt`는 잠시 주석 처리 해두었습니다. 필요할 때 주석을 해제하면 됩니다.

시간대별로 화자를 구분하는 함수 `speaker_diarization` whisper_stt.py

```
(... 생략 ...)
if __name__ == "__main__":
    audio_file_path = "./chap05/audio/싼기타_비싼기타.mp3"           # 원본 오디오 파일
    stt_output_file_path = "./chap05/audio/싼기타_비싼기타.csv"      # STT 결과 파일
    rttm_file_path = "./chap05/audio/싼기타_비싼기타.rttm"          # 화자 분리 원본 파일
    rttm_csv_file_path = "./chap05/audio/싼기타_비싼기타_rttm.csv"   # 화자 분리 CSV 파일

    # result, df = whisper_stt(
    #     audio_file_path,
    #     stt_output_file_path
    # )
    # print(df)

    df_rttm = speaker_diarization(
        audio_file_path,
        rttm_file_path,
        rttm_csv_file_path
    )

    print(df_rttm)
```

이 코드를 실행하면 지정한 경로에 파일이 잘 생성되어 있습니다.

```
chap05 > audio > 📄 싼기타_비싼기타_rttm.csv > 📄 data
1  start,end,speaker_id,duration
2  0.993,30.204,SPEAKER_01,29.21100000000002
3  32.414,42.708,SPEAKER_00,10.293999999999997
4  41.645,44.024,SPEAKER_01,2.378999999999998
5  45.813,67.10900000000001,SPEAKER_00,21.296000000000006
6  67.227,82.786,SPEAKER_01,15.558999999999997
7  84.659,102.564,SPEAKER_00,17.904999999999987
8  103.492,117.532,SPEAKER_01,14.039999999999992
9  119.759,138.6760000000002,SPEAKER_00,18.917000000000016
10 139.351,168.96699999999998,SPEAKER_01,29.615999999999985
11 170.907,192.321,SPEAKER_00,21.413999999999987
12 192.322,193.689,SPEAKER_01,1.3669999999999902
```

통합
화자분리&
whisper

- whisperX 3.8(무료. Local)
 - ffmpeg 못 씀.
 - torch codec 써야 함.
 - numpy 2.1, torch 2.8, pyannote 4.0 이상

gpt-4o-transcribe-diarize(유료)

STT 결과 파일과 화자 분리 결과 파일(RTTM) 결합하기

이제 시간대별로 화자가 분리되었으니 위스퍼 모델이 받아쓴 문장이 어느 시간대의 화자가 말한 것인지 찾아서 정리하면 됩니다.

1. 화자별로 발언을 할당하기 위한 stt_to_rttm 함수를 만듭니다.

시간대별로 화자가 분리된 결과에 STT 결과 텍스트 할당하기

whisper_stt.py

```
(... 생략 ...)  
def stt_to_rttm(  
    audio_file_path: str,  
    stt_output_file_path: str,  
    rttm_file_path: str,  
    rttm_csv_file_path: str,  
    final_output_csv_file_path: str  
):  
  
    result, df_stt = whisper_stt(  
        audio_file_path,  
        stt_output_file_path  
    )  
  
    df_rttm = speaker_diarization(  
        audio_file_path,  
        rttm_file_path,  
        rttm_csv_file_path  
    )  
  
    df_rttm["text"] = ""  
  
    for i_stt, row_stt in df_stt.iterrows():  
        overlap_dict = {}  
        for i_rttm, row_rttm in df_rttm.iterrows():  
            overlap = max(0, min(row_stt["end"], row_rttm["end"]) - max(row_stt["start"],  
row_rttm["start"]))  
            overlap_dict[i_rttm] = overlap  
  
        max_overlap = max(overlap_dict.values())  
        max_overlap_idx = max(overlap_dict, key=overlap_dict.get)  
  
        if max_overlap > 0:  
            df_rttm.at[max_overlap_idx, "text"] += row_stt["text"] + "\n"
```

```

df_rttm.to_csv(
    final_output_csv_file_path,
    index=False, # 인덱스는 저장하지 않음
    sep='|',
    encoding='utf-8'
)
return df_rttm
(... 생략 ...)

```

4

- 1 stt_to_rttm 함수는 whisper_stt 함수와 speaker_diarization 함수를 실행하여 결과를 받아옵니다.
- 2 그 후 가져온 시간대별로 어느 화자가 발언했는지 정리된 df_rttm에 "text"라는 빈 열을 추가하고 위스퍼 모델로 받아온 텍스트를 채웁니다. 중요한 부분은 for 문을 사용해 위스퍼 모델로 받아온 결과인 df_stt의 'start', 'end', 'text' 정보를 이용해 df_rttm의 적절한 행을 찾습니다.
- 3 df_stt 행과 df_rttm 행이 겹치는 시간을 각각 계산해서 가장 많이 겹치는 행에 텍스트를 추가합니다. 겹치는 시간이 0초라면 해당 문장을 건너뛵니다.
- 4 결과는 지정된 경로에 저장됩니다.

2. 이제 파이썬 파일의 메인 부분에서는 stt_to_rttm 함수만 실행하면 되므로, 다음과 같이 기존에 만든 함수들은 모두 주석으로 처리하고 최종 결과를 저장할 파일 경로를 추가합니다.

최종 결과 파일 경로 추가하기

whisper_stt.py

```

(... 생략 ...)
if __name__ == "__main__":
    audio_file_path = "./chap05/audio/싼기타_비싼기타.mp3"
    stt_output_file_path = "./chap05/audio/싼기타_비싼기타.csv"
    rttm_file_path = "./chap05/audio/싼기타_비싼기타.rttm"
    rttm_csv_file_path = "./chap05/audio/싼기타_비싼기타_rttm.csv"
    final_csv_file_path = "./chap05/audio/싼기타_비싼기타_final.csv"
    # result, df = whisper_stt(
    #     audio_file_path,
    #     stt_output_file_path
    # )
    # print(df)

    # df_rttm = speaker_diarization(
    #     audio_file_path,
    #     rttm_file_path,
    #     rttm_csv_file_path
    # )

    # print(df_rttm)

```

```
df_rttm = stt_to_rttm(
    audio_file_path,
    stt_output_file_path,
    rttm_file_path,
    rttm_csv_file_path,
    final_csv_file_path
)

print(df_rttm)
```

이 코드를 실행하면 시간대별로 발화자가 SPEAKER_00, SPEAKER_01로 구분되고 어떤 대화를 했는지 정리됩니다.

	start	end	speaker_id	duration	text
0	0.993	30.204	SPEAKER_01	29.211	지금부터 저랑 역할극을 합니다.\n역할극을 스탠드인 코미디 스타일로 할 건데\n토론...
1	32.414	42.708	SPEAKER_00	10.294	좋습니다. 그럼 제가 썸 기타로 시작하는게 좋다는 입장을 말아볼게요. 그럼 성형님은...
2	41.645	44.024	SPEAKER_01	2.379	네, 됐어요. 시작하시죠.\n
3	45.813	67.109	SPEAKER_00	21.296	좋아요, 먼저 3기타로 시작하는 게 좋은 이유를 말씀드리겠습니다.\n초보자일 때는 ...
4	67.227	82.786	SPEAKER_01	15.559	아, 저는 지금 말에 어폐가 있다고 생각해요.\n왜냐하면 어차피 지금 비싼 기타로...
5	84.659	102.564	SPEAKER_00	17.905	그런데 비싼 기타로 시작하면\n특시라도 품질이 나거나 실수할 때 부담이 더 크지 않...
6	103.492	117.532	SPEAKER_01	14.040	썸 기타를 뭐하러 또 샅니까?\n그리고 기타 실력이랑 품질이랑은 아무 상관이 없어요...
7	119.759	138.676	SPEAKER_00	18.917	하하! 기타를 망치로 치진 않지만 그래도 초보자들은 실수도 많고 조심스럽게 다루기 ...
8	139.351	168.967	SPEAKER_01	29.616	충분하죠. 아 맞아요. 그것도 맞는 말이에요. 썸기타도 요새 품질이\n많이 좋아서...
9	170.907	192.321	SPEAKER_00	21.414	하하\n역시 좋은 기타를 사면 책임감도\n더 커진다는 말씀이시군요. 그래도 비싼 기...
10	192.322	193.689	SPEAKER_01	1.367	NaN
11	192.760	193.503	SPEAKER_00	0.743	NaN

05-4 회의록을 정리하는 AI 서기 완성하기

앞서 분석한 결과를 바탕으로 회의록을 MS 워드로 완성해 보겠습니다. 회의록에는 다음 2가지 내용을 담으려고 합니다.

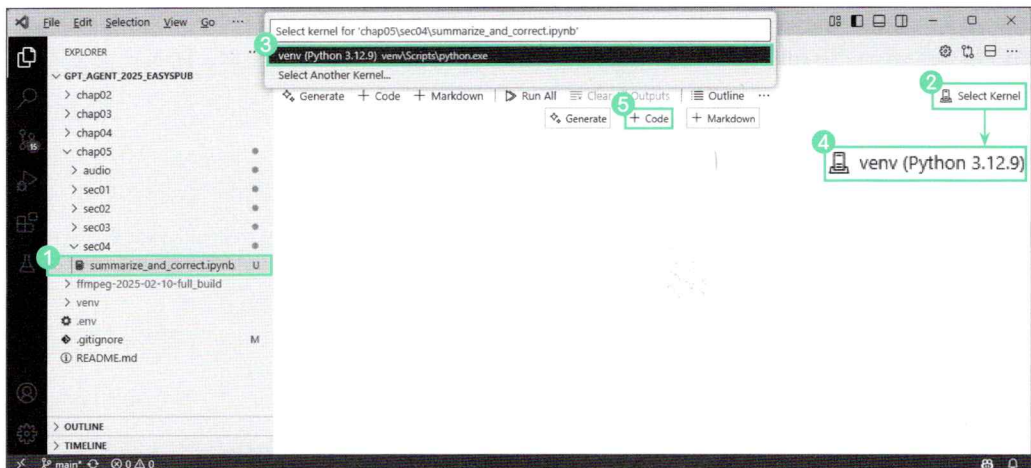
1. 전체 회의 내용 요약
2. 화자별 발언 내용을 순서대로 정리

Do it! 실습 전체 회의 내용 요약하기

■ 결과 파일: sec04/summarize_and_correct.ipynb

앞서 만들었던 CSV 파일에서 SPEAKER_00, SPEAKER_01로 표시되었던 부분에 화자의 이름을 넣고 GPT에게 회의 내용 요약을 요청해 보겠습니다.

1. 단계별로 진행하기 위해 주피터 노트북 파일을 생성합니다. 파일명은 상관없지만 여기에 서는 summarize_and_correct.ipynb로 만들었습니다.



2. 기존에 만들었던 회의록 CSV 파일을 열어 df_rttm 변수에 담습니다.

회의록 CSV 파일 열기

summarize_and_correct.ipynb (1)

```
import pandas as pd

meeting_note_csv_path = "../audio/싼기타_비싼기타_final.csv"
df_rttm = pd.read_csv(meeting_note_csv_path, sep='|')

display(df_rttm)
```

	start	end	speaker_id	duration	text
0	0.993	30.204	SPEAKER_01	29.211	지금부터 저항 역할극을 합니다.\n역할극을 스탠드인 코미디 스타일로 할 건데\n토론...
1	32.414	42.708	SPEAKER_00	10.294	좋습니다. 그럼 제가싼 기타로 시작하는게 좋다는 입장을 말아볼게요. 그럼 성형님은...
2	41.645	44.024	SPEAKER_01	2.379	네, 됐어요. 시작하시죠.\n
3	45.813	67.109	SPEAKER_00	21.296	좋아요. 먼저 3기타로 시작하는 게 좋은 이유를 말씀드리겠습니다.\n초보자일 때는 ...
4	67.227	82.786	SPEAKER_01	15.559	아, 저는 지금 말에 어폐가 있다고 생각해요.\n왜냐하면 어차피 지금 비싼 기타로...
5	84.659	102.564	SPEAKER_00	17.905	그런데 비싼 기타로 시작하면\n혹시라도 흠집이 나거나 실수할 때 부담이 더 크지 않...
6	103.492	117.532	SPEAKER_01	14.040	싼 기타를 뭐하러 또 샅니까?\n그리고 기타 실력이랑 흠집이랑은 아무 상관이 없어요...
7	119.759	138.676	SPEAKER_00	18.917	하하! 기타를 망치로 치진 않지만 그래도 초보자들은 실수도 많고 조심스럽게 다루기 ...
8	139.351	168.967	SPEAKER_01	29.616	충분하죠. 아 맞아요. 그것도 맞는 말이에요. 쌍기타도 요새 품질이\n많이 좋아서...
9	170.907	192.321	SPEAKER_00	21.414	하하\n역시 좋은 기타를 사면 책임감도\n더 커진다는 말씀이시군요. 그래도 비싼 기...
10	192.322	193.689	SPEAKER_01	1.367	NaN
11	192.760	193.503	SPEAKER_00	0.743	NaN

3. SPEAKER_00과 SPEAKER_01로 표시된 부분을 활용해 실제 화자의 이름을 넣어 줍니다.

실제 이름 넣기

summarize_and_correct.ipynb (2)

```
# 이름 넣기
name_dict = {
    "SPEAKER_00": "AI",
    "SPEAKER_01": "이성용",
}

df_rttm["name"] = df_rttm["speaker_id"].apply(lambda x: name_dict[x])

display(df_rttm)
```

	start	end	speaker_id	duration	text	name
0	0.993	30.204	SPEAKER_01	29.211	지금부터 저항 역할극을 합니다.\n역할극을 스탠드인 코미디 스타일로 할 건데\n토론...	이성용
1	32.414	42.708	SPEAKER_00	10.294	좋습니다. 그럼 제가싼 기타로 시작하는게 좋다는 입장을 말아볼게요. 그럼 성형님은...	AI
2	41.645	44.024	SPEAKER_01	2.379	네, 됐어요. 시작하시죠.\n	이성용
3	45.813	67.109	SPEAKER_00	21.296	좋아요. 먼저 3기타로 시작하는 게 좋은 이유를 말씀드리겠습니다.\n초보자일 때는 ...	AI

4. 회의 내용을 요약할 때 꼭 필요하지 않은 항목도 있으므로 `df_rttm`에서 필요한 정보만 뽑아내겠습니다. 그중에서 `start`, `end`, `name`, `text` 항목만 선택하고 이를 JSON 형식으로 변환합니다. 이렇게 변환한 텍스트는 나중에 프롬프트에 추가해서 회의 내용을 요약하는 데 사용합니다.

요약을 위해 필요한 정보만 추출하기

summarize_and_correct.ipynb (3)

```
meeting_note_txt = df_rttm[['start', 'end', 'name', 'text']].to_json(orient='records',
force_ascii=False)
print(meeting_note_txt)
```

```
[{"start":0.993,"end":30.204,"name":"이성용","text":"지금부터 저랑 역할극을 합시다.\n역할극을 스탠드인 코미디 스타일로 할 건데\nt론을 하면서 자연스럽게 대화하는 형식으로 하면서\n코미디를 진행해봅시다.\n그래서 좀 재밌고 자연스럽게 유머러스하게 저랑 대화를 하시면 돼요.\n자연스럽게.\n그리고 주제는 쌍기타로 전기기타를 시작하는 게 좋으냐\n아니면 비싼 기타로 전기기타를 시작하는 게 좋으냐\n이거를 입장을 나눠가지고\n저랑 토론해보면 좋을 것 같아요.\n어, 두 중에 어떤 역할 맡으실래요?\n"},
(... 생략 ...),
{"start":423.644,"end":429.348,"name":"AI","text":"알겠습니다. 언제든 다시 이야기 나누고 싶을 때 편하게 말씀해 주세요. 감사합니다.\n"}]
```

5. GPT에게 회의록을 작성해 달라고 프롬프트를 작성합니다. 프롬프트 아래에 앞에서 JSON 형식으로 만든 문자열 `meeting_note_txt`를 f-string 방식으로 넣어 줍니다.

◆ f-string은 formatted string literal의 줄임말로, 문자열 앞에 `f`를 붙여서 생성합니다. 이 방식은 문자열 내에서 중괄호 `{ }`를 사용해 변수나 표현식을 직접 삽입할 수 있게 합니다.

회의록 작성을 요청하는 프롬프트 작성하기

summarize_and_correct.ipynb (4)

```
system_prompt = f'''
너는 회의 내용을 요약하는 봇이다. 아래 회의록을 읽고, 주요 내용을 요약하라.
결과는 마크다운 형식으로 작성한다.

아래 형식에 맞추어 작성하라.

# 회의 제목

## 주요 내용

## 참석자별 입장

## 결정 사항
```

===== 이하 회의록 =====

```
{ meeting_note_txt }  
...
```

6. GPT에게 요청하기 전에 오픈AI의 API client 를 선언합니다.

◆ GPT를 활용할 때는 보안 문제에 유의해야 합니다.
유출되면 안 되는 기밀 정보가 있다면 로컬 모델을
사용하는 것을 권장합니다.

오픈AI API 설정하기

 summarize_and_correct.ipynb (5)

```
from openai import OpenAI  
from dotenv import load_dotenv  
import os  
  
load_dotenv()  
api_key = os.getenv('OPENAI_API_KEY')  
client = OpenAI(api_key=api_key)
```

7. 이제 GPT 클라이언트를 이용해 프롬프트를 보냅니다. 이 요청에 대한 답신은 **summary**에 답았습니다.

GPT-4o 모델로 회의록 요약 후 출력하기

 summarize_and_correct.ipynb (6)

```
# 오픈AI API를 사용해 요약 결과 생성  
response = client.chat.completions.create(  
    model='gpt-4o',  
    temperature=0.1,  
    messages=[  
        {"role": "system", "content": system_prompt},  
    ]  
)  
  
# 요약 결과를 파일로 저장  
summary = response.choices[0].message.content  
summary = summary.strip() # 좌우 공백 제거  
  
print(summary)
```

이 셀을 실행해 보면 회의 내용이 적절히 잘 요약되었습니다!

기타 구매 시 싼 기타와 비싼 기타 중 어느 것이 더 나은 선택인가?

주요 내용

- 기타를 처음 시작할 때 싼 기타로 시작하는 것이 좋은지, 비싼 기타로 시작하는 것이 좋은지에 대한 토론이 진행되었다.
- 싼 기타로 시작하는 쪽은 초보자에게 부담이 적고, 실수에 대한 걱정이 덜하다는 점을 강조했다.
- 비싼 기타로 시작하는 쪽은 처음부터 좋은 기타를 사용함으로써 이중 지출을 막고, 더 나은 연주 경험을 제공할 수 있다는 점을 강조했다.
- 기타의 품질과 연주 경험, 그리고 개인의 연습 열정이 중요한 요소로 논의되었다.

참석자별 입장

- ****이성용****: 비싼 기타로 시작하는 것이 더 나은 선택이라고 주장. 비싼 기타는 연주할 때 더 좋은 경험을 제공하며, 연습에 대한 동기부여가 될 수 있다고 강조.
- ****AI****: 싼 기타로 시작하는 것이 더 나은 선택이라고 주장. 초보자에게 부담이 적고, 기타에 대한 흥미를 잃지 않도록 도와준다고 강조.

결정 사항

- 기타 구매는 개인의 스타일과 상황에 맞게 선택하는 것이 중요하다는 결론에 도달했다.
- 어떤 기타를 선택하든 꾸준한 연습과 열정이 가장 중요하다는 점에 동의했다.

8. 회의록을 guitar_summary.md로 저장합니다. 내용은 출력된 결과와 동일합니다.



회의 내용 요약 결과 저장하기

summarize_and_correct.ipynb (7)

```
with open('audio/guitar_summary.md', 'w', encoding='utf-8') as f:
    f.write(summary)
```

회의 제목

기타 구매 시 싼 기타와 비싼 기타 중 어느 것이 더 나은 선택인가?

주요 내용

- 기타를 처음 시작할 때 싼 기타로 시작하는 것이 좋은지, 비싼 기타로 시작하는 것이 좋은지에 대한 토론이 진행되었다.
- 싼 기타로 시작하는 쪽은 초보자에게 부담이 적고, 실수에 대한 걱정이 덜하다는 점을 강조했다.
- 비싼 기타로 시작하는 쪽은 처음부터 좋은 기타를 사용함으로써 이중 지출을 막고, 더 나은 연주 경험을 제공할 수 있다는 점을 강조했다.
- 기타의 품질과 연주 경험, 그리고 개인의 연습 열정이 중요한 요소로 논의되었다.

Do it! 실습 GPT로 녹취록 교정하기

■ 결과 파일: sec04/summarize_and_correct.ipynb

앞선 실습에서 만든 CSV 파일을 보면 음성을 잘못 받아 적은 내용이 종종 보입니다. 화자의 발음이 불명확하거나 전문 용어, 사람 이름, 사건 이름 등이 포함되면 실시간으로 받아쓰기가 어려울 수 있습니다. 이번 실습에서는 GPT를 이용해 잘못 기록된 내용을 교정하고 완성된 회의록을 출력해 보겠습니다.

STT 원본을 GPT로 교정하기

앞서 작업하던 `summarize_and_correct.ipynb` 파일에 이어서 진행하겠습니다.

1. 지금까지 사용한 `df_rttm`에서 필요한 항목은 `start`, `end`, `name`, `text`입니다. 이 항목들만 복사하여 새로운 데이터프레임 `df_meeting_note`를 만들고 `text`에 값이 없는 행은 `dropna()` 명령어로 삭제합니다. 그리고 이 데이터프레임을 딕셔너리 형태로 변환하여 `meeting_note_dict` 변수에 저장합니다.

필요한 항목만 추출하여 딕셔너리로 변환하기

■ `summarize_and_correct.ipynb` (8)

```
df_meeting_note = df_rttm[['start', 'end', 'name', 'text']].copy()
df_meeting_note.dropna(inplace=True)

meeting_note_dict = df_meeting_note.to_dict(orient='records') # 딕셔너리 형태로 변환
meeting_note_dict
```

이 출력해 보면 리스트 안에 내용이 딕셔너리 형태로 들어 있는 것을 볼 수 있습니다.

```
[{'start': 0.993,
  'end': 30.204,
  'name': '이성용',
  'text': '지금부터 저랑 역할극을 합시다.\n역할극을 스탠드인 코미디 스타일로 할 건데\n토론을 하면서 자연스럽게 대화하는 형식으로 하면서\n코미디를 진행해봅시다.\n그래서 좀 재밌고 자연스럽게 유머러스하게 저랑 대화를 하시면 돼요.\n자연스럽게.\n그리고 주제는 쌍기타로 전기기타를 시작하는 게 좋으냐\n아니면 비싼 기타로 전기기타를 시작하는 게 좋으냐\n이거를 입장을 나눠가지고\n저랑 토론해보면 좋을 것 같아요.\n어, 두 중에 어떤 역할 맡으실래요?\n'},
 {'start': 32.414,
  'end': 42.708,
  'name': 'AI',
```

```
'text': '좋습니다. 그럼 제가 썸 기타로 시작하는게 좋다는 입장을 맡아볼게요. 그럼 성형님은 비싼 기타로 시작하는 게 좋다는 입장이지요? 네 맞아요. 준비 되셨나요?\n'},
(... 생략 ...)
```

2. 이제 녹취록에 기록된 메시지를 for 문으로 확인하며 GPT에게 수정을 요청합니다.

메시지 하나씩 GPT에게 확인받고 수정하기

summarize_and_correct.ipynb (9)

```
for row in meeting_note_dict:
    if not isinstance(row['text'], str): # 텍스트가 아닌 다른 형식인 경우 통과
        continue

    correction_system_prompt = f'''
        너는 주어진 회의 녹취록을 수정 및 보완하는 봇이다.
        주어진 회의 녹취록은 STT로 작성된 결과이므로, 이 중에 오류가 있는 부분을 찾아내고 수정하라.

        표현, 말투는 최대한 원본과 일치하도록 유지하되, 잘못 받아 적은 내용을 수정하라.
        원본의 내용을 최대한 살리되, 잘못된 내용만 수정하라.
        어떤 경우에는 A라는 사람이 말한 내용을 B가 말한 것으로 잘못 기록된 경우도 있을 수 있다.
        이런 경우에는 문맥을 고려하여 수정하라.

        원본 내용 중 빠지거나, 추가되거나, 잘못 기록된 부분을 찾아내어 수정하라. 수정된 메시지
        이외에는 아무것도 작성하지 말라.

        -----
        회의 요약문 : {summary}
        -----
        회의 녹취록 전문: {df_meeting_note.to_json(orient='records', force_ascii=False)}
        -----
        확인할 메시지 원본: {row['text']}
        ...

    response = client.chat.completions.create(
        model='gpt-4o',
        temperature=0.1,
        messages=[
            {"role": "system", "content": correction_system_prompt},
        ]
    )

    correction = response.choices[0].message.content
    if ':' in correction: # "수정: "과 같은 형식으로 반환되는 경우 해결
        correction = correction.split(':')[1]
```

```
correction = correction.strip() # 좌우 공백 제거

print(row['name'])
print(row['text'])
print(correction)
print('-----')

row['corrected_text'] = correction ④
```

- ❶ 간혹 text에 빈 값이나 숫자가 들어 있을 수 있으므로 is_instance를 사용해 문자열이 아닌 경우에는 통과합니다.
- ❷ correction_system_prompt에는 교정을 위한 요구 사항을 자세히 작성합니다.
- ❸ 각 메시지를 수정할 때 GPT가 그 메시지가 속한 문맥을 이해하고 정확히 수정할 수 있도록 전체 회의 내용 요약과 녹취록 전문을 제공합니다. 회의 녹취록 전문은 df_meeting_note를 .to_json 함수로 변환하여 텍스트로 입력합니다.
- ❹ GPT가 수정안을 제공하면 수정된 내용은 딕셔너리에서 'corrected_text'에 추가됩니다.
- ❺ 종종 '수정:' 처럼 반환될 때가 있는데, 그럴 때는 ':'를 기준으로 분할하고 뒷부분만 사용하도록 해결합니다.

이 셀을 실행해 보면 STT가 잘못 받아 적은 메시지가 수정된 상태로 반환됩니다. '쌍기타'에서 '싼 기타'로 잘 수정되었습니다.

(... 생략 ...)

이성용

아 맞아요.

그 지금 얘기하신 거 보니까

쌍기타가 더 좋다는 얘기는 안 하시네요.

아 맞아요.

그 지금 얘기하신 거 보니까

싼 기타가 더 좋다는 얘기는 안 하시네요.

(... 생략 ...)

이렇게 만든 `meeting_note_dict`를 JSON 파일 형식으로 저장합니다.

결과를 JSON으로 저장하기

summarize_and_correct.ipynb (10)

```
# meeting_note_dict json 파일로 저장
import json

with open('audio/guitar_meeting_note_corrected.json', 'w', encoding='utf-8') as f:
    json.dump(meeting_note_dict, f, ensure_ascii=False, indent=4)
```

이 셀을 실행해 보면 JSON 형식으로 잘 정리되어 저장됩니다.

JSON 형식으로 저장된 결과

guitar_meeting_note_corrected.json

```
[
  {
    "start": 0.993,
    "end": 30.204,
    "name": "이성용",
    "text": "지금부터 저랑 역할극을 합시다.\n역할극을 스탠드인 코미디 스타일로 할 건데\n토론을 하면서 자연스럽게 대화하는 형식으로 하면서\n코미디를 진행해봅시다.\n그래서 좀 재밌고 자연스럽게 유머러스하게 저랑 대화를 하시면 돼요.\n자연스럽게.\n그리고 주제는 쌍기타로 전기기타를 시작하는 게 좋으냐\n아니면 비싼 기타로 전기기타를 시작하는 게 좋으냐\n이거를 입장을 나눠가지고\n저랑 토론해보면 좋을 것 같아요.\n어, 두 중에 어떤 역할 맡으실래요?",
    "corrected_text": "지금부터 저랑 역할극을 합시다.\n역할극을 스탠드업 코미디 스타일로 할 건데\n토론을 하면서 자연스럽게 대화하는 형식으로 하면서\n코미디를 진행해봅시다.\n그래서 좀 재밌고 자연스럽게 유머러스하게 저랑 대화를 하시면 돼요.\n자연스럽게.\n그리고 주제는 쌍 기타로 전기기타를 시작하는 게 좋으냐\n아니면 비싼 기타로 전기기타를 시작하는 게 좋으냐\n이거를 입장을 나눠가지고\n저랑 토론해보면 좋을 것 같아요.\n어, 두 중에 어떤 역할 맡으실래요?"
  },
  {
    "start": 32.414,
    "end": 42.708,
    "name": "AI",
    "text": "좋습니다. 그럼 제가 쌍 기타로 시작하는게 좋다는 입장을 말아볼게요. 그럼 성형님은 비싼 기타로 시작하는 게 좋다는 입장이지요? 네 맞아요. 준비 되셨나요?",
    "corrected_text": "좋습니다. 그럼 제가 쌍 기타로 시작하는 게 좋다는 입장을 말아볼게요. 그럼 성형님은 비싼 기타로 시작하는 게 좋다는 입장이지요? 네 맞아요. 준비 되셨나요?"
  },
  (... 생략 ...)
```

수정한 녹취록을 마크다운 형식과 MS 워드 파일로 출력하기

마지막으로 GPT로 교정한 guitar_meeting_note_corrected.json 파일을 마크다운 형식과 MS 워드 파일로 출력해 보겠습니다.

1. 다음 코드를 입력해 guitar_meeting_note_corrected.json 파일을 마크다운 형식으로 저장합니다. 회의록의 화자 이름과 발언을 마크다운 리스트 항목으로 변환하여 하나의 문자열로 결합합니다.

결과를 마크다운 형식으로 저장하기

summarize_and_correct.ipynb (11)

```
md_template = ""

for row in meeting_note_dict:
    md_template += f"- **{row['name']]** : {row['corrected_text']}\n"

with open('audio/guitar_meeting_note_corrected.md', 'w', encoding='utf-8') as f:
    f.write(md_template)
```

이 코드를 실행해 보면 대화가 마크다운 형식으로 잘 출력됩니다.

Preview guitar_meeting_note_corrected.md X

- 이성용 : 지금부터 저장 역할을 합니다. 역할을 스탠드업 코미디 스타일로 할 건데 토론을 하면서 자연스럽게 대화하는 형식으로 하면서 코미디를 진행해봅시다. 그래서 좀 재밌고 자연스럽게 유머러스하게 저장 대화를 하시면 돼요. 자연스럽게. 그리고 주제는 썬 기타로 전기기타를 시작하는 게 좋으나 아니면 비싼 기타로 전기기타를 시작하는 게 좋으나 이거를 입장을 나눠가지고 저장 토론해보면 좋을 것 같아요. 어, 두 중에 어떤 역할 맡으실래요?
- AI : 좋습니다. 그럼 제가 썬 기타로 시작하는 게 좋다는 입장을 말아볼게요. 그럼 성용님은 비싼 기타로 시작하는 게 좋다는 입장이지요? 네 맞아요. 준비 되셨나요?
- 이성용 : 네, 준비됐어요. 시작하시죠.

마크다운보다 더 대중적인 형식인 MS 워드 파일로 출력할 수도 있습니다. 파이썬에서 MS 워드 파일로 출력하려면 python-docx라는 패키지를 사용합니다.

2. 가상 환경에 python-docx 패키지를 설치합니다.

python-docx 패키지 설치하기

summarize_and_correct.ipynb (12)

```
%pip install python-docx
```

3. 다음은 python-docx 패키지를 이용해 회의 녹취록을 작성하는 코드입니다. 앞서 만든 meeting_note_dict의 내용을 for 문으로 순회하면서 첫 번째 열에는 화자 이름을 넣고 두 번째 열에는 수정한 내용을 넣습니다. 그리고 그 결과를 MS 워드 파일로 저장합니다.

◆ 이 책은 생성형 AI를 활용한 업무 자동화 방법에 초점을 맞추고 있으므로 python-docx를 이용한 문서 스타일 적용 방법 등이 궁금하다면 패키지 문서(<https://python-docx.readthedocs.io/>)를 참고하세요.



회의 내용을 표 형태로 정리한 MS 워드 파일로 출력하기

summarize_and_correct.ipynb (13)

```
from docx import Document

document = Document() # docx 객체 생성
document.add_heading('회의록', level=1) # 제목 추가

table = document.add_table(rows=1, cols=2) # 테이블 생성
table.style = 'Table Grid' # 테이블 스타일 지정

hdr_cells = table.rows[0].cells # 첫 번째 행에 셀 추가
hdr_cells[0].text = 'Speaker' # 첫 번째 셀에 Speaker 추가
hdr_cells[1].text = 'Content' # 두 번째 셀에 Content 추가

# 수정된 녹취록 파일을 읽어 들여서 테이블에 추가
for row in meeting_note_dict:
    row_cells = table.add_row().cells
    row_cells[0].text = row['name']
    row_cells[1].text = row['corrected_text']

document.save('audio/guitar_meeting_note_corrected.docx') # docx 파일 저장
```

이 코드를 실행하면 MS 워드 파일이 생성됩니다. 이제 이 파일에서 세부 편집을 하여 회의록을 깔끔하게 작성한 후 관련자들에게 보내 줄 수 있습니다.

회의록	
Speaker	Content
이성용	지금부터 저랑 역할극을 합니다. 역할극을 스탠드업 코미디 스타일로 할 건데, 토론을 하면서 자연스럽게 대화하는 형식으로 하면서, 코미디를 진행해봅시다. 그래서 좀 재밌고 자연스럽게 유머러스하게 저랑 대화를 하시면 돼요. 자연스럽게. 그리고 주제는 뽀빠리 전기기타를 시작하는 게 좋으나, 아니면 비싼 기타로 전기기타를 시작하는 게 좋으나, 이거를 입장을 나눠가지고, 저랑 토론해보면 좋을 것 같아요. 어, 두 중에 어떤 역할 맡으실래요?
AI	좋습니다. 그럼 제가 뽀빠리 시작하는 게 좋다는 입장을 알아볼게요. 그럼 성용님은 비싼 기타로 시작하는 게 좋다는 입장이시죠? 네 맞아요. 준비 되셨나요?
이성용	네, 준비했어요. 시작하시죠.